

JOBSHEET

PRAKTIKUM BASIS DATA LANJUT

Jurusan Teknologi Informasi
POLITEKNIK NEGERI MALANG
2024



Week 2

SQL SERVER - SELECT, JOIN SORTING DAN FILTERING DATA

Team Teaching:

Habibie Ed Dien, S.Kom., M.T.

Irsyad Arif Mashudi, S.Kom M.Kom

Vit Zuraida, S.Kom., M.Kom.

Rokhimatul Wakhidah, S.Pd., M.T.

Annisa Taufika Firdausi, ST., MT.

Elok Nur Hamdana, S.T., M.T



Information Technology Department, Malang State Polytechnic

Jobsheet- 1 : Introduction to Transact-SQL and Statements SELECT, Join, Sorting, and Filtering data

Advanced Database Course

Supervisor: Advanced Database Teaching Team

September 2024

Topics

1. Introduction to T-SQL and *Query Select*
2. Querying Multiple Tables
3. Sorting and Filtering Data

Objective

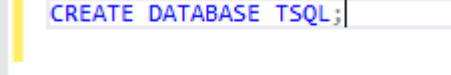
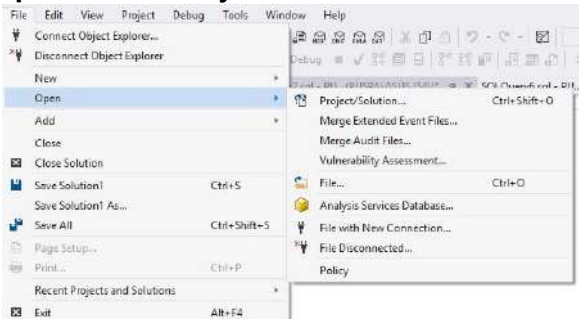
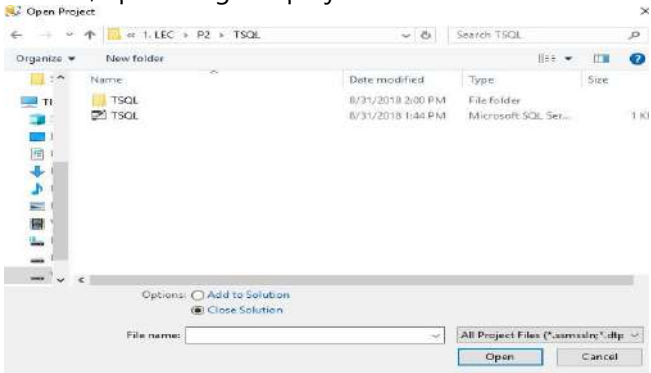
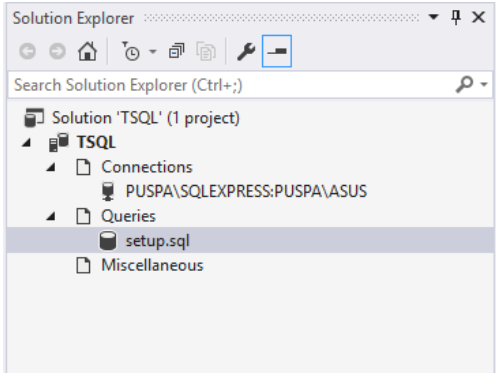
Students are expected to be able to:

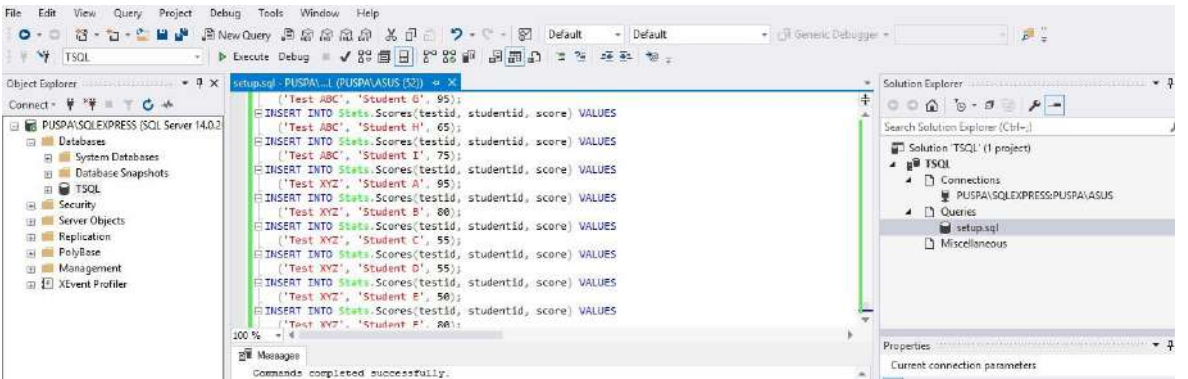
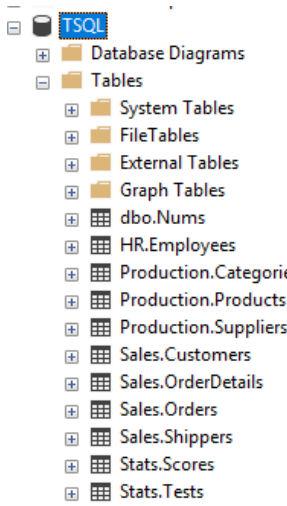
1. Understanding the basic differences between Transact-SQL (T-SQL) and ANSI SQL.
2. Understanding how to create *a database* from an existing SQL file
3. Understand how to execute part or all of a SQL *script* from an existing file.
4. Understanding the concept of using '*comments*' in T-SQL.
5. Understand the concept of using the SELECT statement to analyze existing tables in *a database* .
6. Understanding how to display data in a *unique / distinct manner* .
7. Understand how to use *ALIAS* for table names and column names.
8. Understand the concept of *CASE* expressions and how to use them.
9. Students understand how to query multiple tables in a SELECT clause using JOIN.
10. Students understand how to write INNER JOIN , OUTER JOIN , SELF-JOIN and CROSS JOIN queries .
11. Students understand how to do Data Sorting , Data Filtering with predicates , Data Filtering with TOP and OFFSET-FETCH
12. Students understand how to handle missing and unknown values in real data.

General Instructions

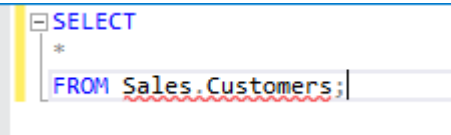
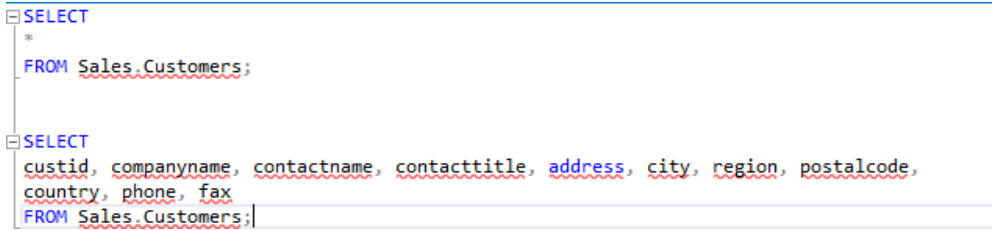
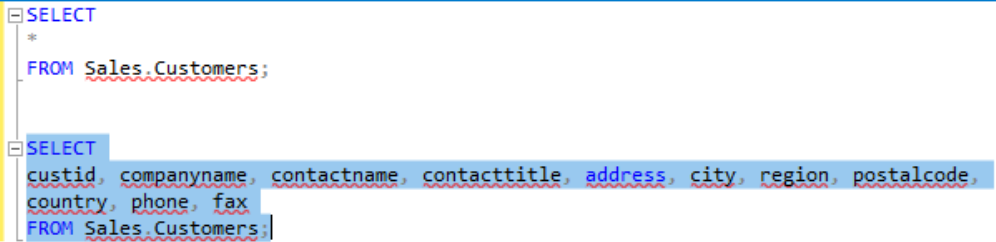
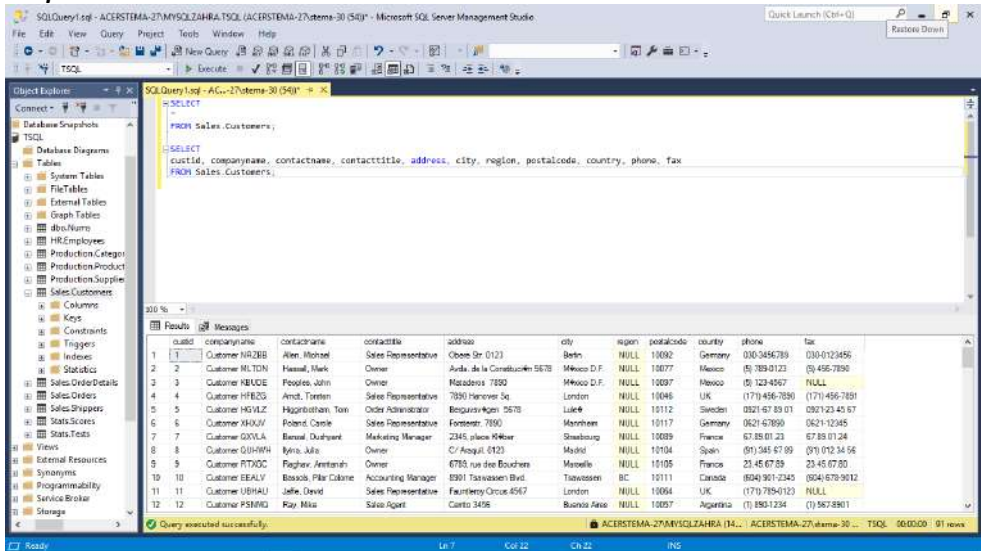
1. Follow the steps in the practical sections in the order given.
2. Answer all questions marked **[Question-X]** that are found in certain steps in each part of the practicum.
3. In each step of the practicum, there is an explanation that will help you answer the questions in instruction number 3, so read and do all the practicum parts in this jobsheet.
4. Write the answers to the questions in the instructions number 3 in a report that is done using a word processing application (Word, OpenOffice, or other similar). Export as a **PDF file** with the following name format:
 - **BDL_Class_03_YourFullName** .pdf
 - Collect the PDF files as a practical report to the supervising lecturer.
 - In addition to the file name, also include your identity on the first page of the report.

Practical Preparation: Creating a Database from Existing SQL

Step	Information
1	Create a TSQL database 
2	On the File menu , click Open and click Project/Solution . 
3	In the Open Project window , open the given project file. 
4	Next, the Solution Explorer window will display the following display. Then please open the “Setup” file. This file contains the sql <i>script</i> to create the tables needed for this practicum. 
5	After the setup file is opened, a display like the image below will appear. Then click <i>Execute</i> and please wait until the process is complete.

																																																	
6	After the process is successful, several tables will be formed, as shown in the image below. 																																																
7	For example, to check <i>records</i> in the Sales.Customers table, please execute the command below: <pre> USE [TSQL] GO SELECT [custid] ,[companyname] ,[contactname] ,[contacttitle] ,[address] ,[city] ,[region] ,[postalcode] ,[country] ,[phone] ,[fax] FROM [Sales].[Customers] GO </pre>																																																
8	The results of the SQL command above are as follows <table border="1"> <thead> <tr> <th></th><th>custid</th><th>companyname</th><th>contactname</th><th>contacttitle</th><th>address</th><th>city</th><th>region</th></tr> </thead> <tbody> <tr> <td>1</td><td>1</td><td>Customer NRZBB</td><td>Allen, Michael</td><td>Sales Representative</td><td>Obere Str. 0123</td><td>Berlin</td><td>NULL</td></tr> <tr> <td>2</td><td>2</td><td>Customer MLTDN</td><td>Hassall, Mark</td><td>Owner</td><td>Avda. de la Constitución 5678</td><td>México D.F.</td><td>NULL</td></tr> <tr> <td>3</td><td>3</td><td>Customer KBUDE</td><td>Peoples, John</td><td>Owner</td><td>Mataderos 7890</td><td>México D.F.</td><td>NULL</td></tr> <tr> <td>4</td><td>4</td><td>Customer HFBZG</td><td>Amdt, Torsten</td><td>Sales Representative</td><td>7890 Hanover Sq.</td><td>London</td><td>NULL</td></tr> <tr> <td>5</td><td>5</td><td>Customer HGVLZ</td><td>Higginbotham, Tom</td><td>Order Administrator</td><td>Berguvsvägen 5678</td><td>Luleå</td><td>NULL</td></tr> </tbody> </table>		custid	companyname	contactname	contacttitle	address	city	region	1	1	Customer NRZBB	Allen, Michael	Sales Representative	Obere Str. 0123	Berlin	NULL	2	2	Customer MLTDN	Hassall, Mark	Owner	Avda. de la Constitución 5678	México D.F.	NULL	3	3	Customer KBUDE	Peoples, John	Owner	Mataderos 7890	México D.F.	NULL	4	4	Customer HFBZG	Amdt, Torsten	Sales Representative	7890 Hanover Sq.	London	NULL	5	5	Customer HGVLZ	Higginbotham, Tom	Order Administrator	Berguvsvägen 5678	Luleå	NULL
	custid	companyname	contactname	contacttitle	address	city	region																																										
1	1	Customer NRZBB	Allen, Michael	Sales Representative	Obere Str. 0123	Berlin	NULL																																										
2	2	Customer MLTDN	Hassall, Mark	Owner	Avda. de la Constitución 5678	México D.F.	NULL																																										
3	3	Customer KBUDE	Peoples, John	Owner	Mataderos 7890	México D.F.	NULL																																										
4	4	Customer HFBZG	Amdt, Torsten	Sales Representative	7890 Hanover Sq.	London	NULL																																										
5	5	Customer HGVLZ	Higginbotham, Tom	Order Administrator	Berguvsvägen 5678	Luleå	NULL																																										

Practical – Part 1: Executing part or all of a SQL script

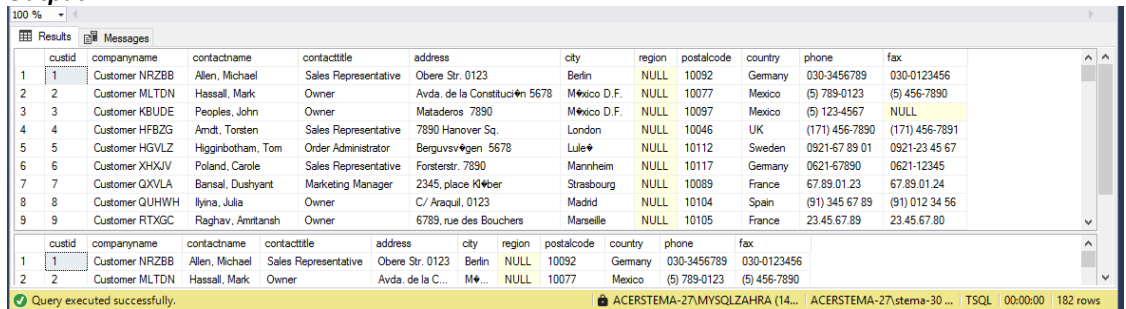
Step	Information
1	Please type the following <i>query in your query panel</i> then click <i>execute</i> . Note the results displayed. 
2	Please add the following <i>query to your query panel</i> then click <i>execute</i> . Note the results displayed 
3	Make a selection on one of the existing <i>queries</i> then click <i>execute</i> . Note the results displayed. What is the difference with the results in the second step above? (Question 1) a. Input 1  b. Output 1 

c. Input 2

```
SELECT *
FROM Sales.Customers;

SELECT custid, companyname, contactname, contacttitle, address, city, region, postalcode, country, phone, fax
FROM Sales.Customers;
```

d. Output 2



	custid	companyname	contactname	contacttitle	address	city	region	postalcode	country	phone	fax
1	1	Customer NRZBB	Allen, Michael	Sales Representative	Obere Str. 0123	Berlin	NULL	10092	Germany	030-3456789	030-0123456
2	2	Customer MLTDN	Hassall, Mark	Owner	Avda. de la Constitución 5678	México D.F.	NULL	10077	Mexico	(5) 789-0123	(5) 456-7890
3	3	Customer KBUDE	Peoples, John	Owner	Mataderos 7890	México D.F.	NULL	10097	Mexico	(5) 123-4567	NULL
4	4	Customer HFBZG	Amdt, Torsten	Sales Representative	7890 Hanover Sq.	London	NULL	10046	UK	(171) 456-7890	(171) 456-7891
5	5	Customer HGVZL	Higginsbotham, Tom	Order Administrator	Berguvsvägen 5678	Luleå	NULL	10112	Sweden	0921-67 89 01	0921-23 45 67
6	6	Customer XHXJV	Poland, Carole	Sales Representative	Forsterstr. 7890	Mannheim	NULL	10117	Germany	0621-67890	0621-12345
7	7	Customer QXVLA	Bansal, Dushyant	Marketing Manager	2345, place Kléber	Strasbourg	NULL	10089	France	67.89.01.23	67.89.01.24
8	8	Customer QUHWH	Ilyina, Julia	Owner	C/ Araquil, 0123	Madrid	NULL	10104	Spain	(91) 345 67 89	(91) 012 34 56
9	9	Customer RTXGC	Raghav, Amitansh	Owner	6789, rue des Bouchers	Marseille	NULL	10105	France	23.45.67.89	23.45.67.80

	custid	companyname	contactname	contacttitle	address	city	region	postalcode	country	phone	fax
1	1	Customer NRZBB	Allen, Michael	Sales Representative	Obere Str. 0123	Berlin	NULL	10092	Germany	030-3456789	030-0123456
2	2	Customer MLTDN	Hassall, Mark	Owner	Avda. de la C...	Méx...	NULL	10077	Mexico	(5) 789-0123	(5) 456-7890

Query executed successfully. ACERSTEMA-27\MYSQLZAHRA (14... ACERSTEMA-27\stema-30 ... TSQL 00:00:00 182 rows

The two SQL queries you provided have the following differences in their results:

1. Column Selection:

- **First Query:** `SELECT * FROM Sales.Customers;`
 - This query retrieves all columns from the `Sales.Customers` table. The result set will include every column that exists in the table, regardless of how many columns there are.
- **Second Query:** `SELECT custid, companyname, contactname, contacttitle, address, city, region, postalcode, country, phone, fax FROM Sales.Customers;`
 - This query explicitly selects and retrieves only the columns listed (i.e., `custid, companyname, contactname, etc.`). The result set will include only these specific columns, even if the table contains more columns.

2. Flexibility and Explicitness:

- **First Query:** It is less explicit since it retrieves all columns. This is useful if you want to see every column without specifying them. However, it can lead to inefficiency if the table has many columns and you only need a few.
- **Second Query:** It is more explicit and controlled, allowing you to retrieve only the columns you are interested in. This is generally better for performance and clarity, especially in cases where the table has many columns, but you only need specific data.

In summary, the main difference is that the first query retrieves all columns, while the second query retrieves only the specified columns.

4

In the query panel please type

```
SELECT *
FROM
```




5

then on the Object Explorer tab – Tables please find the Sales.Customers table. Click the table and drag it to the query pane I. The result is as shown below, after that add a semicolon after the name of the table in question and click execute.

```
SELECT *  
FROM  
[Sales].[Customers];
```

1. Input & output

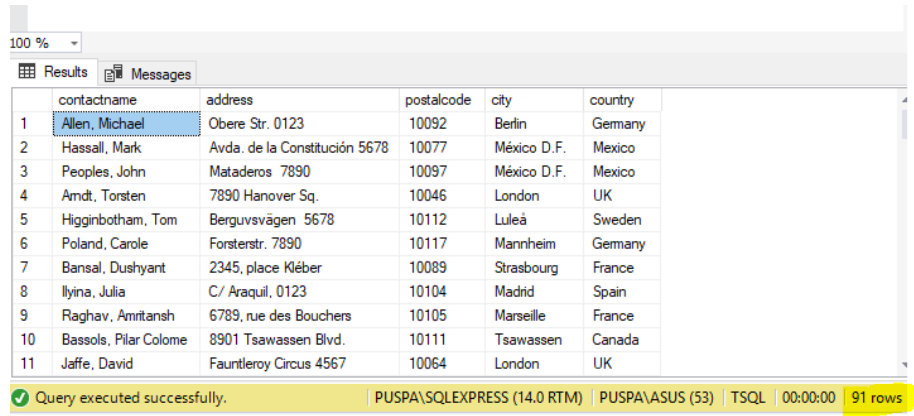
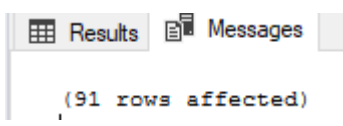
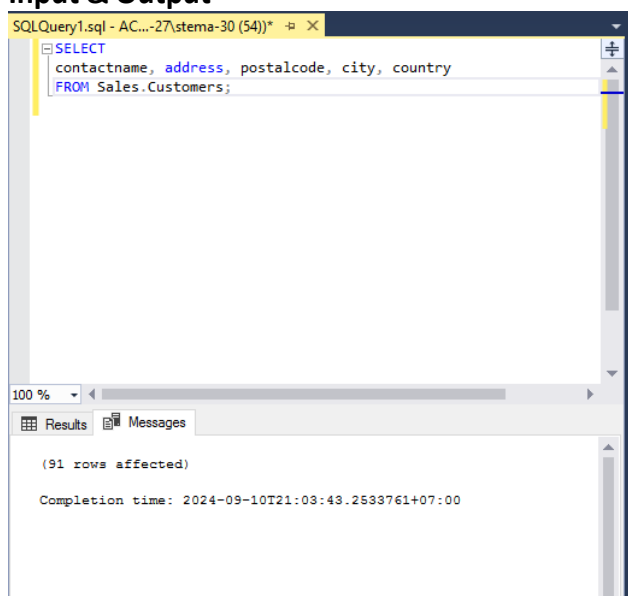
The screenshot shows a SQL Server Enterprise Manager interface. At the top, a query window titled 'SQLQuery1.sql - AC...-27\stema-30 (54))*' contains the following SQL query:

```
SELECT *  
FROM  
[Sales].[Customers];
```

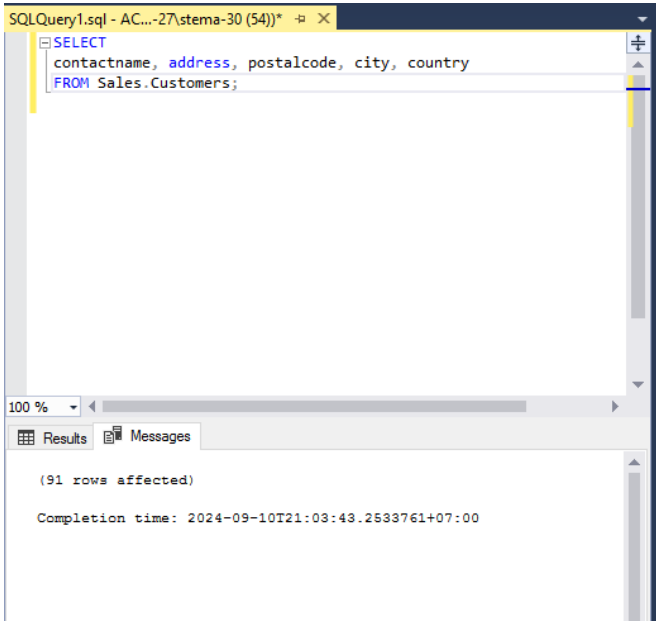
Below the query window, the 'Results' tab is active, displaying a table with 11 rows of data. The columns are: custid, companyname, contactname, contacttitle, and address. The first row is highlighted.

	custid	companyname	contactname	contacttitle	address
1	1	Customer NRZBB	Allen, Michael	Sales Representative	Obere Str. 01
2	2	Customer MLTDN	Hassall, Mark	Owner	Avda. de la C
3	3	Customer KBUDE	Peoples, John	Owner	Mataderos 7
4	4	Customer HFBZG	Amdt, Torsten	Sales Representative	7890 Hanov
5	5	Customer HGVLZ	Higginbotham, Tom	Order Administrator	Berguvsväge
6	6	Customer XHXJV	Poland, Carole	Sales Representative	Forsterstr. 78
7	7	Customer QXVLA	Bansal, Dushyant	Marketing Manager	2345, place I
8	8	Customer QUHWH	Ilyina, Julia	Owner	C/ Araquil, 0
9	9	Customer RTXGC	Raghav, Amritansh	Owner	6789, rue de
10	10	Customer EEALV	Bassols, Pilar Colome	Accounting Manager	8901 Tsawa
11	11	Customer UBHAU	Jaffe, David	Sales Representative	Fauntleroy Ci

Practical – Part 2: Using the SELECT statement for specific columns

Step	Information
1	In the query panel, please type the script below <pre>SELECT contactname, address, postalcode, city, country FROM Sales.Customers;</pre>
2	Highlights query above and click <i>execute</i>
3	Please observe the results. How many <i>rows</i> are produced? To find out, you can do it on the results tab as shown in the image below  Or you can also go to the messages tab as shown in the image below.  a. Input & Output 

Practical – Part 3: Using the SELECT statement to display data *uniquely* / *DISTINCT*

Step	Information
1	In the query panel, please type the script below <pre>SELECT country FROM Sales.Customers;</pre>
2	Highlights query above and click <i>execute</i>
3	Please observe the results. <i>Is there any duplicate data? If YES, why? Capture the results of executing the SQL script above (Question 2)</i> a. Input & Output  Yes, there is duplicate data in the output. The duplicates occur because the query is selecting the country column from the Sales.Customers table, and multiple customers can be from the same country. This results in the same country appearing multiple times in the output.
4	In the query pane, please type the script below. <pre>SELECT DISTINCT country FROM Sales.Customers;</pre> Please click <i>execute</i> and observe the results.



5

*Is there any duplicate data? Explain the difference in results in step 4 and step 3!? What are the benefits of the **DISTINCT** command? Capture the results of executing the SQL script above (Question 3)*

a. Input & Output

The screenshot shows a SQL query window with the following text:

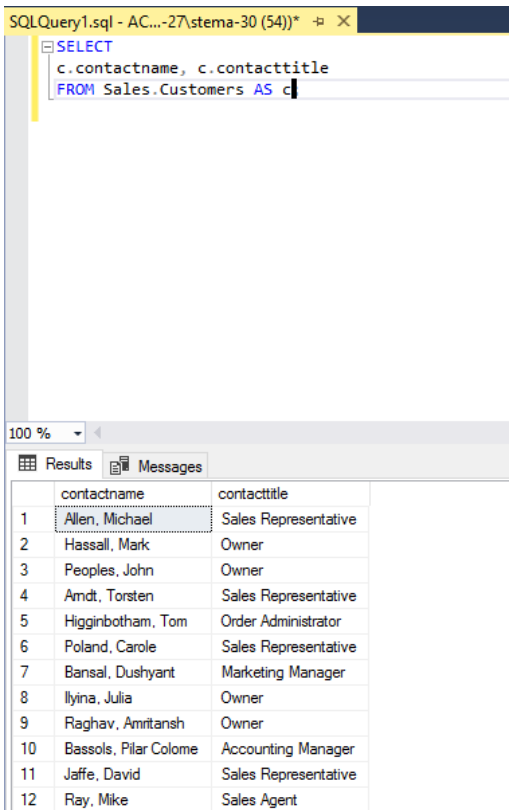
```
SQLQuery1.sql - AC...-27\stema-30 (54))* -> X
SELECT DISTINCT
country
FROM Sales.Customers;
```

Below the query window, the 'Results' tab is active, displaying a table with the following data:

	country
1	Argentina
2	Austria
3	Belgium
4	Brazil
5	Canada
6	Denmark
7	Finland
8	France
9	Germany
10	Ireland
11	Italy

No, there is no duplicate data in the output of the query `SELECT DISTINCT country FROM Sales.Customers;`. The `DISTINCT` keyword ensures that only unique values are returned, so each country appears only once in the result set.

Practical – Part 4: Using *ALIAS* for table names and column names

Step	Information																																							
1	In <i>the query panel</i>, please type <i>the script</i> below <pre>SELECT c.contactname, c.contacttitle FROM Sales.Customers AS c;</pre>																																							
2	<i>Highlights query</i> above and click <i>execute</i>. Observe the results  <table><thead><tr><th></th><th>contactname</th><th>contacttitle</th></tr></thead><tbody><tr><td>1</td><td>Allen, Michael</td><td>Sales Representative</td></tr><tr><td>2</td><td>Hassall, Mark</td><td>Owner</td></tr><tr><td>3</td><td>Peoples, John</td><td>Owner</td></tr><tr><td>4</td><td>Amdt, Torsten</td><td>Sales Representative</td></tr><tr><td>5</td><td>Higginbotham, Tom</td><td>Order Administrator</td></tr><tr><td>6</td><td>Poland, Carole</td><td>Sales Representative</td></tr><tr><td>7</td><td>Bansal, Dushyant</td><td>Marketing Manager</td></tr><tr><td>8</td><td>Ilyina, Julia</td><td>Owner</td></tr><tr><td>9</td><td>Raghav, Amritansh</td><td>Owner</td></tr><tr><td>10</td><td>Bassols, Pilar Colome</td><td>Accounting Manager</td></tr><tr><td>11</td><td>Jaffe, David</td><td>Sales Representative</td></tr><tr><td>12</td><td>Ray, Mike</td><td>Sales Agent</td></tr></tbody></table>		contactname	contacttitle	1	Allen, Michael	Sales Representative	2	Hassall, Mark	Owner	3	Peoples, John	Owner	4	Amdt, Torsten	Sales Representative	5	Higginbotham, Tom	Order Administrator	6	Poland, Carole	Sales Representative	7	Bansal, Dushyant	Marketing Manager	8	Ilyina, Julia	Owner	9	Raghav, Amritansh	Owner	10	Bassols, Pilar Colome	Accounting Manager	11	Jaffe, David	Sales Representative	12	Ray, Mike	Sales Agent
	contactname	contacttitle																																						
1	Allen, Michael	Sales Representative																																						
2	Hassall, Mark	Owner																																						
3	Peoples, John	Owner																																						
4	Amdt, Torsten	Sales Representative																																						
5	Higginbotham, Tom	Order Administrator																																						
6	Poland, Carole	Sales Representative																																						
7	Bansal, Dushyant	Marketing Manager																																						
8	Ilyina, Julia	Owner																																						
9	Raghav, Amritansh	Owner																																						
10	Bassols, Pilar Colome	Accounting Manager																																						
11	Jaffe, David	Sales Representative																																						
12	Ray, Mike	Sales Agent																																						
3	In <i>the query panel</i>, please type <i>the script</i> below. <pre>SELECT c.contactname AS Name, c.contacttitle AS Title, c.companyname AS [Company Name] FROM Sales.Customers AS c;</pre>																																							
4	<i>Highlights query</i> above and click <i>execute</i> . Observe the results.																																							
5	<i>What is the difference between the execution results of the query stage 1 and stage 3 above? What are the benefits of the AS command? Please explain! Capture the results of the SQL script execution above (Question 4)</i>																																							



SQLQuery1.sql - AC...-27\stema-30 (54))*

```
SELECT  
c.contactname AS Name, c.contacttitle AS Title, c.companyname AS  
FROM Sales.Customers AS c;
```

100 %

Results Messages

	Name	Title	Company Name
1	Allen, Michael	Sales Representative	Customer NRZBB
2	Hassall, Mark	Owner	Customer MLTDN
3	Peoples, John	Owner	Customer KBUDE
4	Amdt, Torsten	Sales Representative	Customer HFBZG
5	Higginbotham, Tom	Order Administrator	Customer HGVLZ
6	Poland, Carole	Sales Representative	Customer XHXJV
7	Bansal, Dushyant	Marketing Manager	Customer QXVLA
8	Ilyina, Julia	Owner	Customer QUHWH
9	Raghav, Amritansh	Owner	Customer RTXGC
10	Bassols, Pilar Colome	Accounting Manager	Customer EEALV
11	Jaffe, David	Sales Representative	Customer UBHAU
12	Ray, Mike	Sales Agent	Customer PSNMQ

Difference Between the Execution Results of Query Stage 1 and Stage 3:

1. **Stage 1 Query:**

- **Columns Selected:** This query selects only two columns: `contactname` and `contacttitle`.
- **Output:** The result will display the names of the contacts and their titles from the `Sales.Customers` table. The columns in the output will retain their original names (`contactname` and `contacttitle`).

2. **Stage 3 Query:**

- **Columns Selected:** This query selects three columns: `contactname`, `contacttitle`, and `companyname`.

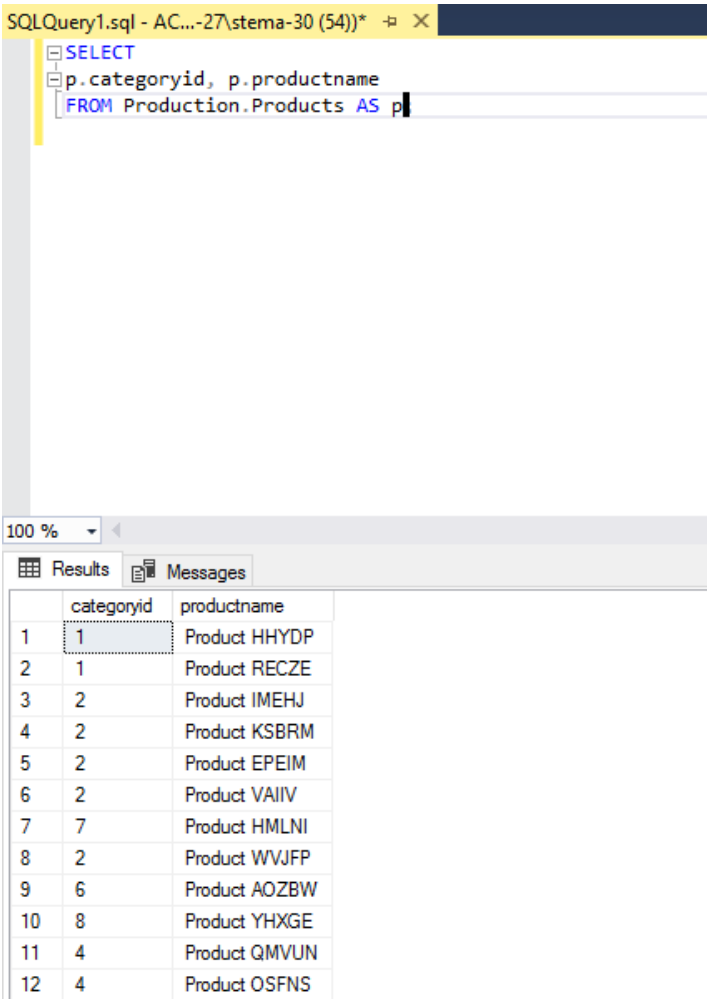


- **Column Aliases:** It uses the `AS` command to rename the columns in the result set:
 - `contactname` is renamed to `Name`
 - `contacttitle` is renamed to `Title`
 - `companyname` is renamed to `Company Name`
- **Output:** The result will display the names of the contacts, their titles, and the names of their companies. The column names in the output will be `Name`, `Title`, and `Company Name`.

Benefits of the `AS` Command:

1. **Improves Readability:**
 - The `AS` command allows you to assign more descriptive or user-friendly names to columns in the result set. This makes the output easier to understand, especially when the original column names are not intuitive.
2. **Custom Labeling:**
 - `AS` helps in customizing the labels of columns in the result set to match the specific needs of reports or data analysis, without altering the original table structure.
3. **Avoids Naming Conflicts:**
 - When combining multiple tables or performing complex queries, `AS` can help avoid naming conflicts by allowing you to create unique aliases for columns that might otherwise have the same name.
4. **Simplifies Queries:**
 - `AS` can shorten long or complex column names, making queries less cumbersome and easier to write and maintain.

Practicum – Part 5: Use of CASE

Step	Information																																							
1	In the query panel, please type the script below <pre>SELECT p.categoryid, p.productname FROM Production.Products AS p;</pre>																																							
2	Highlights query above and click <i>execute</i>. Observe the results  The screenshot shows the SQL query being executed in a query window. Below the query window, the Results pane displays the output of the query. The Results pane has tabs for 'Results' and 'Messages'. The 'Results' tab is active, showing a table with two columns: 'categoryid' and 'productname'. The table contains 12 rows of data. <table><thead><tr><th></th><th>categoryid</th><th>productname</th></tr></thead><tbody><tr><td>1</td><td>1</td><td>Product HHYDP</td></tr><tr><td>2</td><td>1</td><td>Product RECZE</td></tr><tr><td>3</td><td>2</td><td>Product IMEHJ</td></tr><tr><td>4</td><td>2</td><td>Product KSBRM</td></tr><tr><td>5</td><td>2</td><td>Product EPEIM</td></tr><tr><td>6</td><td>2</td><td>Product VAIIV</td></tr><tr><td>7</td><td>7</td><td>Product HMLNI</td></tr><tr><td>8</td><td>2</td><td>Product WVJFP</td></tr><tr><td>9</td><td>6</td><td>Product AOZBW</td></tr><tr><td>10</td><td>8</td><td>Product YHXGE</td></tr><tr><td>11</td><td>4</td><td>Product QMVUN</td></tr><tr><td>12</td><td>4</td><td>Product OSFNS</td></tr></tbody></table>		categoryid	productname	1	1	Product HHYDP	2	1	Product RECZE	3	2	Product IMEHJ	4	2	Product KSBRM	5	2	Product EPEIM	6	2	Product VAIIV	7	7	Product HMLNI	8	2	Product WVJFP	9	6	Product AOZBW	10	8	Product YHXGE	11	4	Product QMVUN	12	4	Product OSFNS
	categoryid	productname																																						
1	1	Product HHYDP																																						
2	1	Product RECZE																																						
3	2	Product IMEHJ																																						
4	2	Product KSBRM																																						
5	2	Product EPEIM																																						
6	2	Product VAIIV																																						
7	7	Product HMLNI																																						
8	2	Product WVJFP																																						
9	6	Product AOZBW																																						
10	8	Product YHXGE																																						
11	4	Product QMVUN																																						
12	4	Product OSFNS																																						



3

In the query panel, please type the script below.

```
SELECT
  p.categoryid, p.productname,
  CASE
    WHEN p.categoryid = 1 THEN 'Beverages'
    WHEN p.categoryid = 2 THEN 'Condiments'
    WHEN p.categoryid = 3 THEN 'Confections'
    WHEN p.categoryid = 4 THEN 'Dairy Products'
    WHEN p.categoryid = 5 THEN 'Grains/Cereals'
    WHEN p.categoryid = 6 THEN 'Meat/Poultry'
    WHEN p.categoryid = 7 THEN 'Produce'
    WHEN p.categoryid = 8 THEN 'Seafood'
    ELSE 'Other'
  END AS categoryname
FROM Production.Products AS p;
```

4

Highlights query above and click *execute* . Observe the results.

5

Highlights query above and click *execute*. Observe the results

SQLQuery1.sql - AC...-27\stema-30 (54))* -p X

```
SELECT
  p.categoryid, p.productname
FROM Production.Products AS p
```

100 %

	categoryid	productname
1	1	Product HHYDP
2	1	Product RECZE
3	2	Product IMEHJ
4	2	Product KSBRM
5	2	Product EPEIM
6	2	Product VAIIV
7	7	Product HMLNI
8	2	Product WVJFP
9	6	Product AOZBW
10	8	Product YHXGE
11	4	Product QMVUN
12	4	Product OSFNS



Difference Between the Execution Results of Query Stage 1 and Stage 3:

1. Stage 1 Query:

- **Columns Selected:** This query selects two columns: `categoryid` and `productname`.
- **Output:** The result will display the category ID and the product name from the `Production.Products` table. The `categoryid` will be a numeric value representing the category, but it won't provide any descriptive information about what the category ID stands for.

2. Stage 3 Query:

- **Columns Selected:** This query selects three columns: `categoryid`, `productname`, and an additional column `categoryname` created by the `CASE` command.
- **Output:** The result will display the category ID, product name, and a descriptive name for the category based on the value of `categoryid`. The `categoryname` column provides a human-readable label for each category, making the data more understandable.

Benefits of the `CASE` Command:

1. Conditional Logic:

- The `CASE` command allows you to implement conditional logic directly within your SQL queries. It enables you to return different values based on specific conditions, similar to if-else statements in programming languages.

2. Data Transformation:

- `CASE` can be used to transform or categorize data on the fly. For instance, it can convert numerical codes (like category IDs) into descriptive labels, making the result set more meaningful without altering the underlying data.

3. Custom Output:

- It allows you to create customized output columns that adapt to the data being queried, enabling dynamic reporting and more tailored results.

4. Handling Nulls and Defaults:

- `CASE` can handle null values and provide default values when none of the specified conditions are met, ensuring that your result set is complete and consistent.

6

In the query panel, please type the script below.



```
SELECT
  p.categoryid, p.productname,
  CASE
    WHEN p.categoryid = 1 THEN 'Beverages'
    WHEN p.categoryid = 2 THEN 'Condiments'
    WHEN p.categoryid = 3 THEN 'Confections'
    WHEN p.categoryid = 4 THEN 'Dairy Products'
    WHEN p.categoryid = 5 THEN 'Grains/Cereals'
    WHEN p.categoryid = 6 THEN 'Meat/Poultry'
    WHEN p.categoryid = 7 THEN 'Produce'
    WHEN p.categoryid = 8 THEN 'Seafood'
    ELSE 'Other'
  END AS categoryname,
  CASE
    WHEN p.categoryid IN (1, 7, 8) THEN 'Campaign Products'
    ELSE 'Non-Campaign Products'
  END AS iscampaing
FROM Production.Products AS p;
```

7

Please capture the results, what data is obtained from the query command above? Explain (Question 6)

The screenshot shows a SQL query window with the following text:

```
SELECT
  p.categoryid, p.productname,
  CASE
    WHEN p.categoryid = 1 THEN 'Beverages'
    WHEN p.categoryid = 2 THEN 'Condiments'
    WHEN p.categoryid = 3 THEN 'Confection'
    WHEN p.categoryid = 4 THEN 'Dairy Products'
    WHEN p.categoryid = 5 THEN 'Grains/Cereals'
    WHEN p.categoryid = 6 THEN 'Meat/Poultry'
    WHEN p.categoryid = 7 THEN 'Produce'
    WHEN p.categoryid = 8 THEN 'Seafood'
    ELSE 'Other'
  END AS categoryname,
  CASE
    WHEN p.categoryid IN (1, 7, 8) THEN 'Campaign Products'
    ELSE 'Non-Campaign Products'
  END AS iscampaing
FROM Production.Products AS p;
```

Below the query, the 'Results' tab is active, displaying a table with 5 columns: categoryid, productname, categoryname, and iscampaing. The table contains 11 rows of data.

	categoryid	productname	categoryname	iscampaing
1	1	Product HHYDP	Beverages	Campaign Products
2	1	Product RECZE	Beverages	Campaign Products
3	2	Product IMEHJ	Condiments	Non-Campaign Products
4	2	Product KSBRM	Condiments	Non-Campaign Products
5	2	Product EPEIM	Condiments	Non-Campaign Products
6	2	Product VAIIV	Condiments	Non-Campaign Products
7	7	Product HMLNI	Produce	Campaign Products
8	2	Product WVJFP	Condiments	Non-Campaign Products
9	6	Product AOZBW	Meat/Poultry	Non-Campaign Products
10	8	Product YHXGE	Seafood	Campaign Products
11	4	Product QMVLIN	Dairy Products	Non-Campaign Products

The query provides a detailed output that categorizes products into their respective categories with meaningful names (categoryname) and also labels them as either part of a "Campaign" or "Non-Campaign" based on specific category IDs.



8

Based on question number 6, please display data that is in the 'seafood' category only and use the *ALIAS command* to change the column name as shown in the image below.

Capture your SQL command and how many rows are produced (Question 7)

	ID_KATEGORI	NAMA_PRODUK	NAMA_KATEGORI	STATUS
1	8	Product ACRVI	Seafood	Campaign Products
2	8	Product AQOKR	Seafood	Campaign Products
3	8	Product CBRRL	Seafood	Campaign Products
4	8	Product CKEDC	Seafood	Campaign Products
5	8	Product EVFFA	Seafood	Campaign Products
6	8	Product GMKIJ	Seafood	Campaign Products
7	8	Product LYERX	Seafood	Campaign Products
8	8	Product POXFU	Seafood	Campaign Products
9	8	Product TTEEX	Seafood	Campaign Products

Answer :

SQLQuery1.sql - AC...-27\stema-30 (54))*

```

SELECT
  p.categoryid AS categoryid,
  p.productname AS productname,
  CASE
    WHEN p.categoryid = 1 THEN 'Beverages'
    WHEN p.categoryid = 2 THEN 'Condiments'
    WHEN p.categoryid = 3 THEN 'Confection'
    WHEN p.categoryid = 4 THEN 'Dairy Products'
    WHEN p.categoryid = 5 THEN 'Grains/Cereals'
    WHEN p.categoryid = 6 THEN 'Meat/Poultry'
    WHEN p.categoryid = 7 THEN 'Produce'
    WHEN p.categoryid = 8 THEN 'Seafood'
    ELSE 'Other'
  END AS productname,
  CASE
    WHEN p.categoryid IN (1, 7, 8) THEN 'Campaign Products'
    ELSE 'Non-Campaign Products'
  END AS STATUS
FROM Production.Products AS p
WHERE
  p.categoryid = 8;

```

100 %

Results Messages

	categoryid	productname	productname	STATUS
1	8	Product YHXGE	Seafood	Campaign Products
2	8	Product POXFU	Seafood	Campaign Products
3	8	Product CKEDC	Seafood	Campaign Products
4	8	Product LYERX	Seafood	Campaign Products
5	8	Product GMKIJ	Seafood	Campaign Products
6	8	Product EVFFA	Seafood	Campaign Products
7	8	Product YZIXQ	Seafood	Campaign Products
8	8	Product TTEEX	Seafood	Campaign Products
9	8	Product AQOKR	Seafood	Campaign Products

ERSTEMA-27\MYSQLZAHRA (14... | ACERSTEMA-27\stema-30 ... | TSQL | 00:00:00 | 12 rows



9

Display employee data from HR.Employees table that comes from country 'USA' and city 'Seattle', use ALIAS command to change column name as shown below. **Capture your SQL command (Question 8)**

	FIRST_NAME	LAST_NAME	CITY	COUNTRY
1	Sara	Davis	Seattle	USA
2	Maria	Cameron	Seattle	USA

The screenshot shows the SQL Developer interface. The top pane displays the following SQL query:

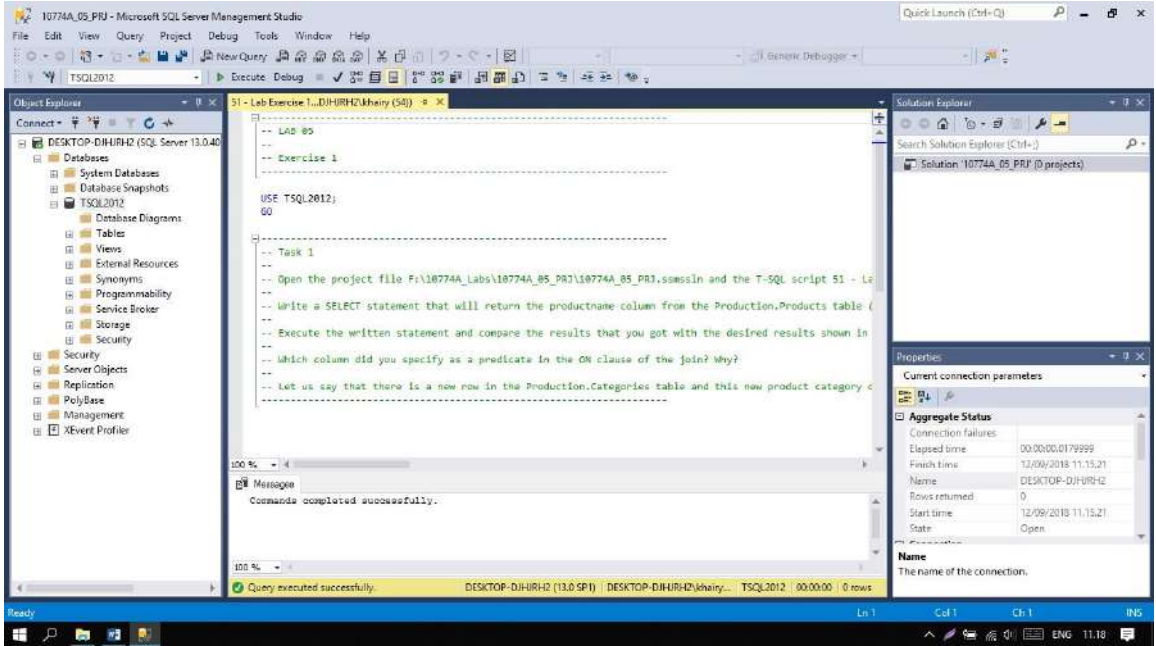
```
SELECT
  e.firstName AS First_Name,
  e.lastName AS Last_Name,
  e.city AS City,
  e.country AS Country
FROM
  HR.Employees AS e
WHERE
  e.country = 'USA'
AND e.city = 'Seattle'
```

The bottom pane shows the query results in a table:

	First_Name	Last_Name	City	Country
1	Sara	Davis	Seattle	USA
2	Maria	Cameron	Seattle	USA

The status bar at the bottom indicates: CERSTEMA-27\MYSQLZAHRA (14... | ACERSTEMA-27\stema-30 ... | TSQL | 00:00:00 | 2 rows

Practical – Part 6 : Creating an Inner Join Query

Step	Information
1	To experiment on this jobsheet, first log in to SQL Server Management Studio (SSMS). Then open the project \10774A Labs\10774A_05_PRJ\10774A_05_PRJ.ssmssln and the T-SQL script 51 - Lab Exercise 1.sql. Make sure the database is connected to “TSQL”.
	
2	[Question- 9] Write a T-SQL SELECT that will display the productname column from the Production.Products table (use the alias table "p") and the categoryname column from the Production.Categories table (use the alias table "c") using inner join.



The screenshot shows the Microsoft SQL Server Management Studio interface. The Object Explorer on the left displays the database structure for 'ACERSTEMA-27\MYSQLZAH'. The main window shows a T-SQL query in 'SQLQuery2.sql' with the following code:

```
SELECT
    p.productname,
    c.categoryname
FROM
    Production.Products AS p
INNER JOIN
    Production.Categories AS c
ON
    p.CategoryID = c.CategoryID;
```

The Results pane at the bottom shows the output of the query, displaying 12 rows of data:

	productname	categoryname
1	Product HHYDP	Beverages
2	Product RECZE	Beverages
3	Product IMEHJ	Condiments
4	Product KSBRM	Condiments
5	Product EPEIM	Condiments
6	Product VAIIV	Condiments
7	Product HMLNI	Produce
8	Product WVJFP	Condiments
9	Product AOZBW	Meat/Poultry
10	Product YHXGE	Seafood
11	Product QMVUN	Dairy Products
12	Product OSFNS	Dairy Products

The status bar at the bottom indicates '77 rows'.

Compare the results in step 2 with the file 52 - Lab Exercise 1 - Task 1 Result.txt. If they are the same then the T-SQL you wrote is correct.

3

The screenshot shows a text file named '52 - Lab Exercise 1 - Task 1 Result.txt' with the following content:

```
productname          categoryname
-----
Product HHYDP        Beverages
Product RECZE        Beverages
Product IMEHJ        Condiments
...
...
Product BWRLG        Beverages
Product JYGFE        Beverages
Product LUNZZ        Condiments

(77 row(s) affected)
```

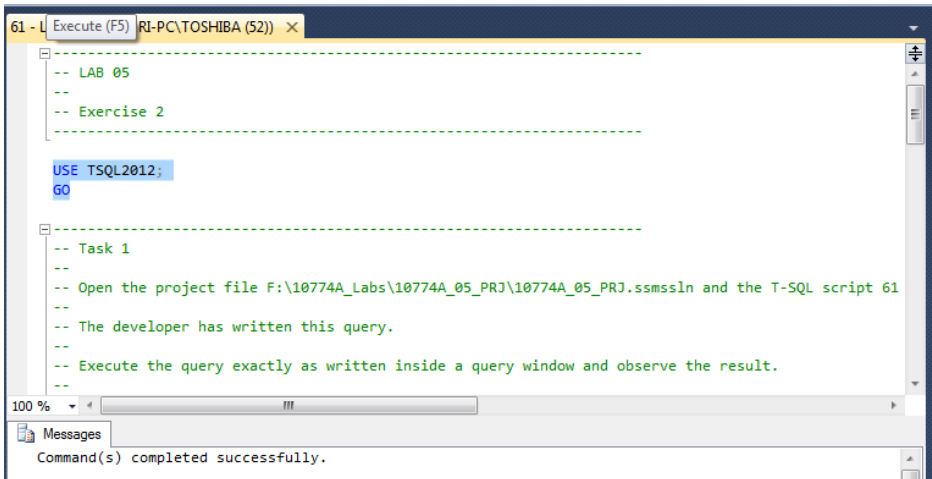
4

[Question- 10] Which column is specified as a predicate in the ON join clause? Why?
the column p .CategoryID from the Production .Products table and c .CategoryID from the Production .Categories table are specified as predicates. The predicate in the ON clause is used to determine how the rows from the two tables should be matched.
Coz the columns used in the ON clause are specified as predicates to define the criteria for how rows from the two tables should be matched and combined in the result set.

5

Conclusion : After carrying out this part of the practicum, students know and understand how to perform an INNER JOIN on two tables.

Practical – Part 7 : Creating an Inner Join Query on Multiple Tables

Step	Information
1	A <i>developer</i> will often be asked to run T-SQL files obtained from various departments . For example, the sales department wants a sales report of all customers for at least one order , with detailed information about each order. Then <i>the developer</i> will prepare the initialization of the SELECT statement to retrieve the custid and contactname columns in the Sales.Orders table. In accordance with the case study, this part 2 practicum will be carried out. Open the project \10774A Labs\10774A_05_PRJ\10774A_05_PRJ.ssmssl and the T-SQL script 61 - Lab Exercise 2.sql. Make sure the database is connected with “TSQL”. 
2	<i>The developer</i> will write T-SQL: <pre>SELECT custid , contactname , orderid FROM Sales . Customers INNER JOIN Sales . Orders ON Customers . custid = Orders . custid ;</pre> Execute the T-SQL , and observe the results!
3	[Question- 11] After the 2nd stage of the experiment is carried out, an error will appear. What is the content of the error message? Why can this error occur? Explain!



```
-- The developer has written this query.
-- Execute the query exactly as written inside a query window.
-- You get an error. What is the error message? Why do you get this error?

SELECT
    custid, contactname, orderid
FROM Sales.Customers
INNER JOIN Sales.Orders ON Customers.custid = Orders.custid

-- Task 2
```

100 %

Messages

Msg 209, Level 16, State 1, Line 23
Ambiguous column name 'custid'.

Completion time: 2024-09-10T22:44:58.5915754+07:00

100 %

MA-27\MYSQLZAHRA (14... | ACERSTEMA-27\stema-30 ... | TSQL | 00:00:00 | 0 rows

indicates that the SQL query contains a column name (`custid`) that is ambiguous. This means that SQL Server cannot determine which table the column `custid` belongs to because it exists in more than one table or has been referenced in multiple contexts without clear qualification.

Why This Error Occurs

1. **Multiple Tables with Same Column Name:** If your query involves multiple tables that all have a column named `custid` and you reference `custid` without specifying which table it belongs to, SQL Server will raise this error. This is because SQL Server does not know which `custid` column to use in the query.
2. **Missing Table Alias:** When you join multiple tables and use columns that have the same name, you need to qualify the column names with their respective table aliases to resolve the ambiguity.

4

[Question- 12] In this 4th trial, fix the error that occurred in the 3rd stage trial which explains that all table names have their own table identities.



61 - Lab Exercise 2...A-27\stema-30 (58))*

```
SELECT
    c.custid,
    c.contactname,
    o.orderid
FROM
    sales.customers AS c
INNER JOIN
    sales.orders AS o
ON
    c.custid = o.custid
ORDER BY
    c.custid, o.orderid;
```

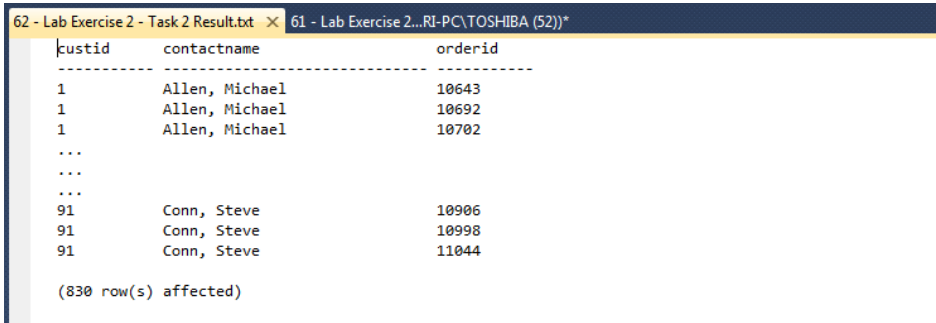
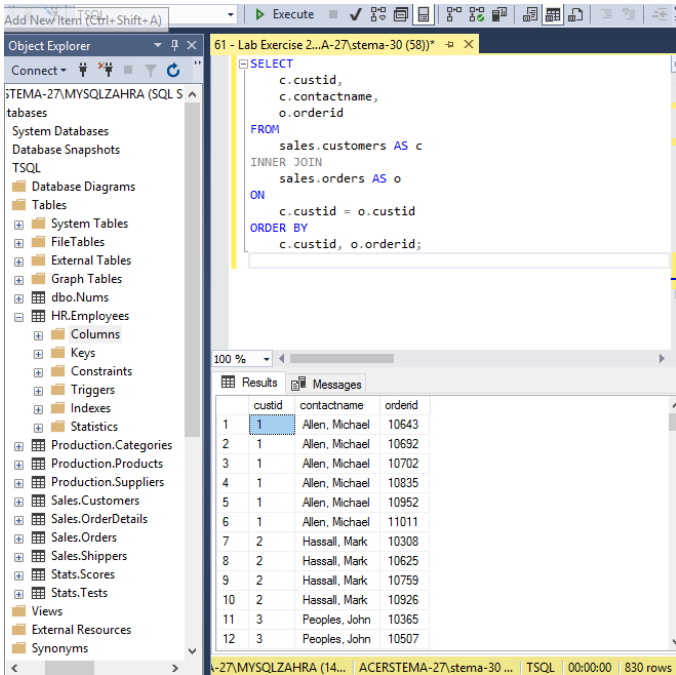
Results

	custid	contactname	orderid
1	1	Allen, Michael	10643
2	1	Allen, Michael	10692
3	1	Allen, Michael	10702
4	1	Allen, Michael	10835
5	1	Allen, Michael	10952
6	1	Allen, Michael	11011
7	2	Hassall, Mark	10308
8	2	Hassall, Mark	10625
9	2	Hassall, Mark	10759
10	2	Hassall, Mark	10926
11	3	Peoples, John	10365
12	3	Peoples, John	10507

TSQL | 00:00:00 | 830 rows

5

Observe and compare the results of the 4th stage trial with the file 62 - Lab Exercise 2 - Task 2 Result.txt. If the results are the same, then your answer is correct.

	
6	[Question- 13] Copy the T-SQL in the 4th stage of the test and modify it by using the alias table " c " to Sales.Customers table and " o " for Sales.Orders table. 
7	Execute T-SQL on stage-6 test and compare the result with the result of stage 4 execution! If the result is the same then your T-SQL is correct.
	Change the column prefix in the SELECT clause to the full name, then execute the T-SQL!

8

```

62 - Lab Exercise 2 - Task 2 Result.txt  61 - Lab Exercise 2...RI-PC\TOSHIBA (52)* X
--
-- Notice that there are full source table names written as table aliases.
-- Apply the needed changes to the SELECT statement so that it will run without an error. Test the c
-- Observe and compare the results that you got with the recommended result shown in the file 62 - L
-----
SELECT
Customers.custid, Customers.contactname, Orders.orderid
FROM Sales.Customers AS c
INNER JOIN Sales.Orders AS o ON c.custid = o.custid;

-- Task 3
-- Copy the T-SQL statement from task 2 and modify it to use the table aliases "C" for the Sales.Cus

```

Messages

```

Msg 209, Level 16, State 1, Line 23
Ambiguous column name 'custid'.
Msg 4104, Level 16, State 1, Line 41
The multi-part identifier "Customers.custid" could not be bound.
Msg 4104, Level 16, State 1, Line 41
The multi-part identifier "Customers.contactname" could not be bound.
Msg 4104, Level 16, State 1, Line 41
The multi-part identifier "Orders.orderid" could not be bound.

```

9

[Question- 14] Why does the execution result of T-SQL stage 8 produce an error?

```

61 - Lab Exercise 2...A-27\stema-30 (58)* X
-- Execute the written statement and compare the results th
-----
SELECT
Customers,custid, Customers.contactname, Orders.orderid
FROM
Sales.Customers AS c
INNER JOIN
Sales.Orders AS o
ON
c.custid = o.custid;

```

Messages

```

Msg 207, Level 16, State 1, Line 66
Invalid column name 'Customers'.
Msg 209, Level 16, State 1, Line 66
Ambiguous column name 'custid'.
Msg 4104, Level 16, State 1, Line 66
The multi-part identifier "Customers.contactname" could not
Msg 4104, Level 16, State 1, Line 66
The multi-part identifier "Orders.orderid" could not be bound
Completion time: 2024-09-10T23:03:12.4987510+07:00

```

The error in your T-SQL query is due to a syntax issue in the `SELECT` clause. Specifically, there is a missing dot (.) between the table name and column names, and incorrect placement of the table alias. Here's a breakdown of the problems and how to fix them:

Issues in the Query

- 1. Incorrect Column Reference:** In the `SELECT` clause, `Customers,custid` is not valid. It should be `Customers.custid` if you are referring to the `custid` column from the `Customers` table.
- 2. Missing Table Alias in SELECT Clause:** You are using table aliases (`c` and `o`) but not



utilizing them correctly in the `SELECT` clause.

10

[Question- 15] Change the column name prefix in the T-SQL test step 8 with its alias name, then display the execution results!

```
-- Execute the written statement and compare the results to the previous results

SELECT
    c.custid,
    c.contactname,
    o.orderid
FROM
    Sales.Customers AS c
INNER JOIN
    Sales.Orders AS o
ON
    c.custid = o.custid;
```

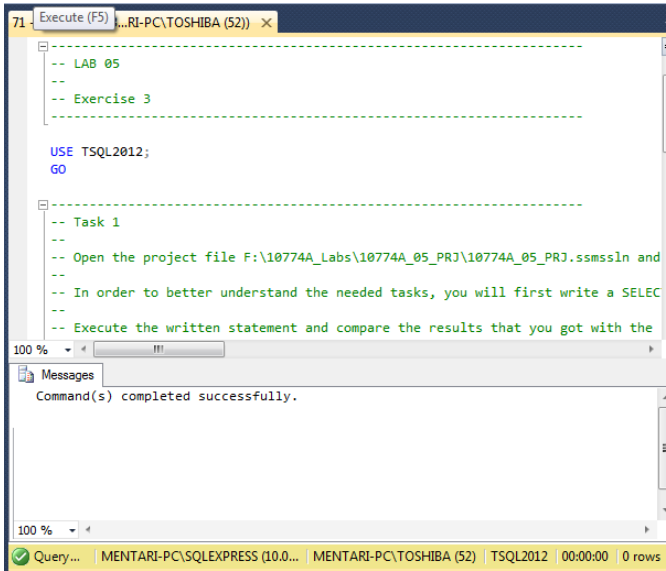
	custid	contactname	orderid
3	1	Allen, Michael	10702
4	1	Allen, Michael	10835
5	1	Allen, Michael	10952
6	1	Allen, Michael	11011
7	2	Hassall, Mark	10308
8	2	Hassall, Mark	10625
9	2	Hassall, Mark	10759
10	2	Hassall, Mark	10926
11	3	Peoples, John	10365
12	3	Peoples, John	10507
13	3	Peoples, John	10535
14	3	Peoples, John	10573

27\MYSQLZAHRA (14... | ACERSTEMA-27\stema-30 ... | TSQL | 00:00:00 | 830 rows

11

Conclusion : After carrying out this part of the practicum, you should now know and understand the importance of using table alias names and how to JOIN multiple tables (more than two tables).

Practical – Part 8 : Creating a Self-Join Query

Step	Information
1	This practicum uses a case study in an HR department that wants to display reports on employees and managers. Some of the things that want to be displayed are the lastname, firstname, and title columns of the HR.Employees table for employees and managers. Open the project \10774A Labs\10774A_05_PRJ\10774A_05_PRJ.ssmssln and the T-SQL script 71 - Lab Exercise 3.sql. Make sure the database is connected with "TSQL". 
2	[Question- 16] Write T-SQL using SELECT clause to display empid, lastname, firstname, title, and mgrid columns. on the table HR.Employees by giving the alias name "e" for the HR.Employees table. <pre> SELECT e.empid, e.lastname, e.firstname, e.title, e.mgrid FROM HR.Employees AS e; </pre>
3	[Question- 17] Execute the 2nd stage of the test and compare it with 72 - Lab Exercise 3 - Task 1 Result.txt . If the results are the same, then your test is correct.



71 - Lab Exercise 3...A-27\stema-30 (58))

```
-- In order to better understand the needed tasks, you will
-- Execute the written statement and compare the results to
```

```
SELECT
  e.empid,
  e.lastname,
  e.firstname,
  e.title,
  e.mgrid
FROM
  HR.Employees AS e;
```

100 %

Results Messages

	empid	lastname	firstname	title	mgrid
1	1	Davis	Sara	CEO	NULL
2	2	Funk	Don	Vice President, Sales	1
3	3	Lew	Judy	Sales Manager	2
4	4	Peled	Yael	Sales Representative	3
5	5	Buck	Sven	Sales Manager	2
6	6	Suurs	Paul	Sales Representative	5
7	7	King	Russell	Sales Representative	5
8	8	Cameron	Maria	Sales Representative	3
9	9	Dolgopyatova	Zoya	Sales Representative	5

4

[Question- 18] Copy the T-SQL in step 2 then modify it by adding columns about manager information, namely lastname, firstname using SELF-JOIN. Use the aliases mgrlastname and mgrfirstname to distinguish the names of managers and employees.

```
SELECT
  e.empid,
  e.lastname AS emplastname,
  e.firstname AS empfirstname,
  e.title,
  e.mgrid,
  mgr.lastname AS mgrlastname,
  mgr.firstname AS mgrfirstname
FROM
  HR.Employees AS e
LEFT JOIN
  HR.Employees AS mgr
ON
  e.mgrid = mgr.empid;
```

5

[Question- 19] Execute the 2nd stage of the test and compare it with 7 3 - Lab Exercise 3 - Task 2 Result.txt . If the results are the same, then your test is correct.



71 - Lab Exercise 3...A-27\stema-30 (58))

```
SELECT
    e.empid,
    e.lastname,
    e.firstname,
    e.title,
    e.mgrid,
    mgr.lastname AS mgrlastname,
    mgr.firstname AS mgrfirstname
FROM
    HR.Employees AS e
LEFT JOIN
    HR.Employees AS mgr
ON
    e.mgrid = mgr.empid;
```

100 %

Results Messages

	empid	lastname	firstname	title	mgrid	mgrlastname
1	1	Davis	Sara	CEO	NULL	NULL
2	2	Funk	Don	Vice President, Sales	1	Davis
3	3	Lew	Judy	Sales Manager	2	Funk
4	4	Peled	Yael	Sales Representative	3	Lew
5	5	Buck	Sven	Sales Manager	2	Funk
6	6	Suurs	Paul	Sales Representative	5	Buck
7	7	King	Russell	Sales Representative	5	Buck
8	8	Cameron	Maria	Sales Representative	3	Lew
9	9	Dolgopyatova	Zoya	Sales Representative	5	Buck

6

[Question- 20] Is it mandatory to write the table alias name when executing the SELF-JOIN command? Can the original table name be used as an alias name? Explain!

While SQL Server allows you to use the original table name as an alias, using clear and distinct table aliases in a SELF-JOIN is considered best practice. It enhances readability, reduces ambiguity, and makes maintaining and understanding your SQL code easier.

Using table aliases in a SELF-JOIN is not strictly mandatory, but it is highly recommended for clarity and correctness. Here's why:

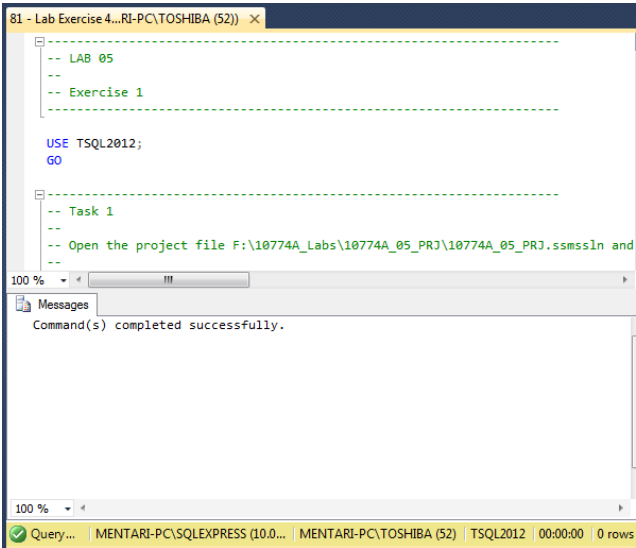
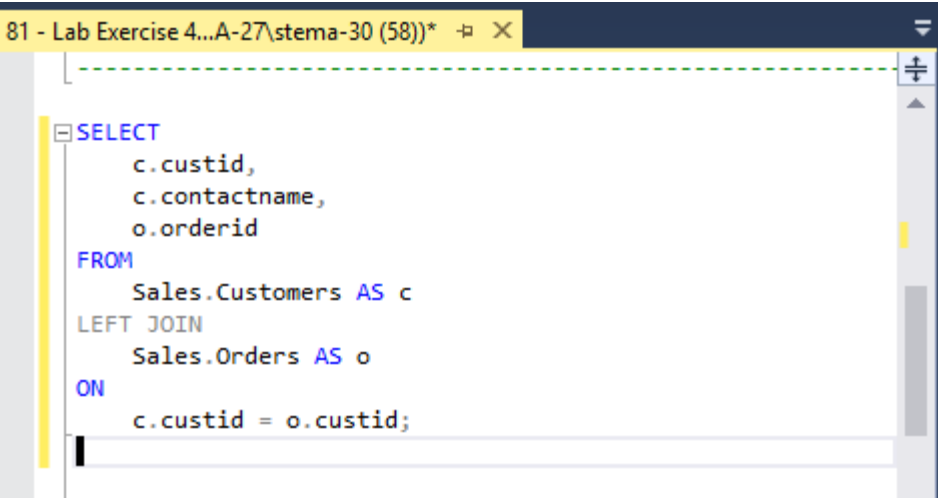
Why Table Aliases are Important in SELF-JOIN

1. **Clarity:** When you perform a SELF-JOIN, you are joining a table with itself. Without table aliases, it can be confusing to distinguish which instance of the table you're referring to, especially when selecting columns or specifying join conditions.
2. **Avoid Ambiguity:** Aliases help avoid ambiguity in column names. If the table has multiple columns with the same name, using aliases makes it clear which table the column belongs to.
3. **Readability:** Aliases make the query more readable and easier to understand. They simplify the query, especially when dealing with multiple tables or complex joins.



	Using the Original Table Name as an Alias Technically, you can use the original table name as an alias, but it is not recommended due to potential confusion. SQL Server allows you to use the same name for both the table and the alias, but this can lead to unclear or ambiguous code.
7	Conclusion : After doing this part of the practicum, you should understand how to write a T-SQL SELF-JOIN statement.

Practical – Part 9 : Creating Outer-Join Query

Step	Information
1	The case study used in this practicum part 4 continues the practicum in part 3. The sales department is quite satisfied with the report that has been made. Then the sales department wants to change the report to show all customers, even though the customer does not have an order history or customers who have an order history. Therefore, a SELECT clause is needed to retrieve all rows from the Sales.Customers table (custid and contactname columns) and the orderid column From the Sales.Orders table. Open the project \10774A Labs\10774A_05_PRJ\10774A_05_PRJ.ssmssln and the T-SQL script 81 - Lab Exercise 4.sql. Make sure the database is connected with "TSQL". 
2	[Question- 21] Write a T-SQL command with a SELECT clause to retrieve the custid and contactname columns from the table Sales.Customers and the orderid column from the Sales.Orders table . The command created must retrieve all rows from the Sales.Customers table . 



3

[Question- 22] Execute the 2nd stage of the test and compare it with 82 - Lab Exercise 4 - Task 1 Result.txt . If the results are the same, then your test is correct.

The screenshot shows a SQL Server Enterprise Manager window titled "81 - Lab Exercise 4...A-27\stema-30 (58))*". The query window displays the following SQL code:

```
SELECT
    c.custid,
    c.contactname,
    o.orderid
FROM
    Sales.Customers AS c
LEFT JOIN
    Sales.Orders AS o
ON
    c.custid = o.custid;
```

Below the query window, the "Results" tab is active, showing a table with 4 columns: "custid", "contactname", and "orderid". The table contains 12 rows of data:

	custid	contactname	orderid
821	90	Larsson, Kat...	10695
822	90	Larsson, Kat...	10873
823	90	Larsson, Kat...	10879
824	90	Larsson, Kat...	10910
825	90	Larsson, Kat...	11005
826	91	Conn, Steve	10374
827	91	Conn, Steve	10611
828	91	Conn, Steve	10792
829	91	Conn, Steve	10870
830	91	Conn, Steve	10906
831	91	Conn, Steve	10998
832	91	Conn, Steve	11044

At the bottom of the window, the status bar indicates: "A-27\MYSQLZAHRA (14... | ACERSTEMA-27\stema-30 ... | TSQL | 00:00:00 | 832 rows".

4

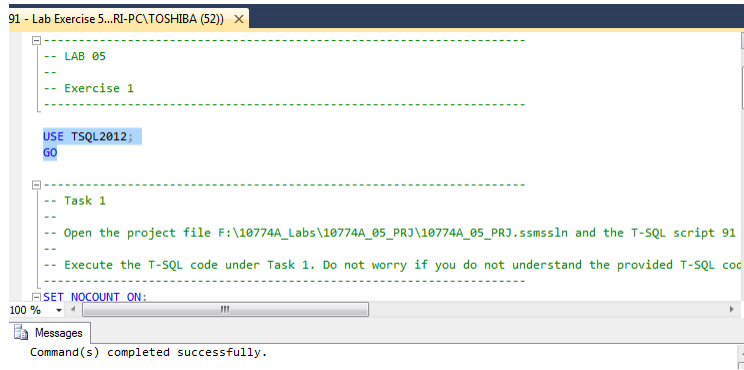
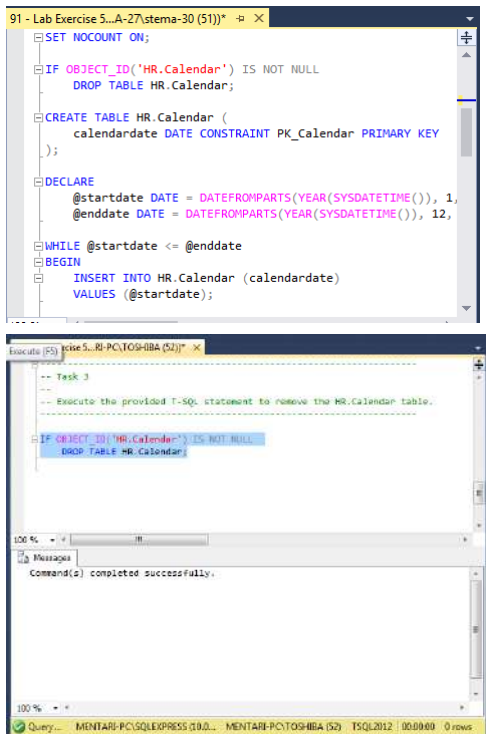
[Question- 23] Pay attention to the values in the `orderid` column . Are there any missing values (NULL)? Why?

Yes, there can be missing values (NULL) in the `orderid` column due to the nature of the `LEFT JOIN`. These `NULL` values appear when there is no matching order for a given customer. This behavior is expected and indicates that some customers do not have corresponding orders in the `Sales.Orders` table.

5

Conclusion : After doing this part of the practicum, you should understand how to write the T-SQL OUTER-JOIN statement .

Practical – Part 10 : Creating a Cross-Join Query

Step	Information
1	This case study begins with the HR department wanting to set up a personal calendar for each employee. The IT department will provide a T-SQL code that generates all days in the past year. Therefore, <i>the developer</i> will use the SELECT clause to return all rows from the calendar table for each row in the HR.Employees table. Open the project \10774A Labs\10774A_05_PRJ\10774A_05_PRJ.ssmssln and the T-SQL script 91 - Lab Exercise 5.sql. Make sure the database is connected with “TSQL”.
	
2	[Question- 24] Run the T-SQL code under task 1. Display the output! (Don't worry if you don't understand the T-SQL code. The next step will provide a more concrete example of how CROSS-JOIN is implemented.) 



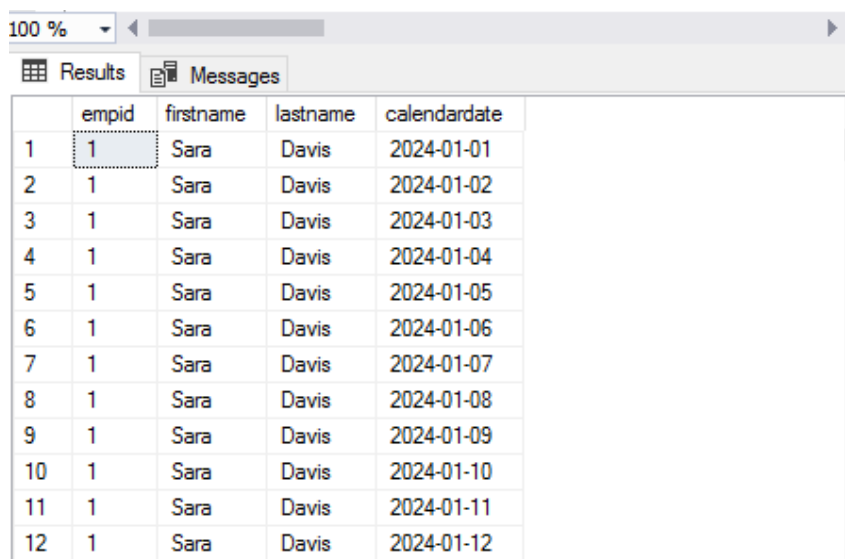
3

[Question- 25] Write a SELECT command to retrieve values from the empid, firstname, and lastname columns from the HR.Employees table and the calendardate column from the HR.Calendar table

```
L
GO
-- observe the HR.Calendar table
SELECT
    e.empid,
    e.firstname,
    e.lastname,
    c.calendardate
FROM
    HR.Employees AS e
CROSS JOIN
    HR.Calendar AS c;
```

4

[Question-2 6] Execute the 3rd stage test and compare it with the file 92 - Lab Exercise 5 - Task 2 Result.txt . If the results are the same, then your test is correct.



100 %

Results Messages

	empid	firstname	lastname	calendardate
1	1	Sara	Davis	2024-01-01
2	1	Sara	Davis	2024-01-02
3	1	Sara	Davis	2024-01-03
4	1	Sara	Davis	2024-01-04
5	1	Sara	Davis	2024-01-05
6	1	Sara	Davis	2024-01-06
7	1	Sara	Davis	2024-01-07
8	1	Sara	Davis	2024-01-08
9	1	Sara	Davis	2024-01-09
10	1	Sara	Davis	2024-01-10
11	1	Sara	Davis	2024-01-11
12	1	Sara	Davis	2024-01-12

5

Drop the HR.Calendar table by executing the T-SQL code below task 3.



```
SET NOCOUNT OFF;

GO
-- observe the HR.Calendar table
SELECT
    e.empid,
    e.firstname,
    e.lastname,
    c.calendardate
FROM
    HR.Employees AS e
CROSS JOIN
    HR.Calendar AS c;
IF OBJECT_ID('HR.Calendar') IS NOT NULL
    DROP TABLE HR.Calendar;
```

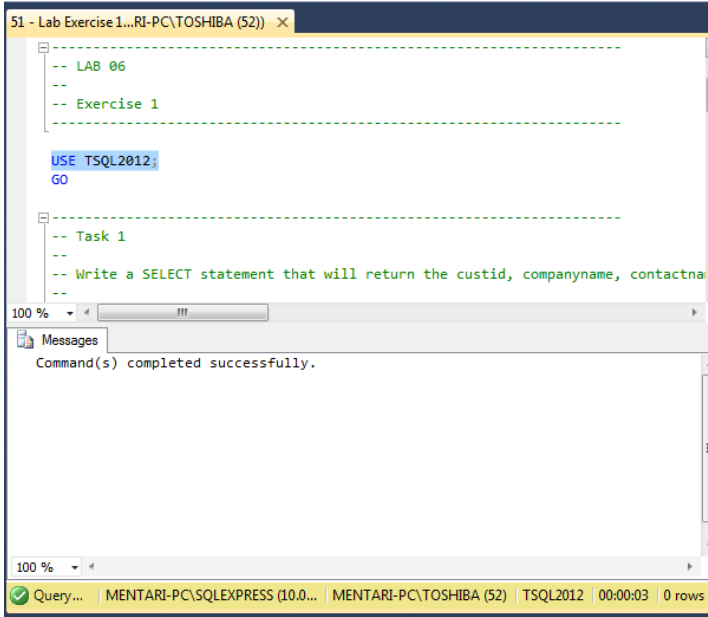
10 %

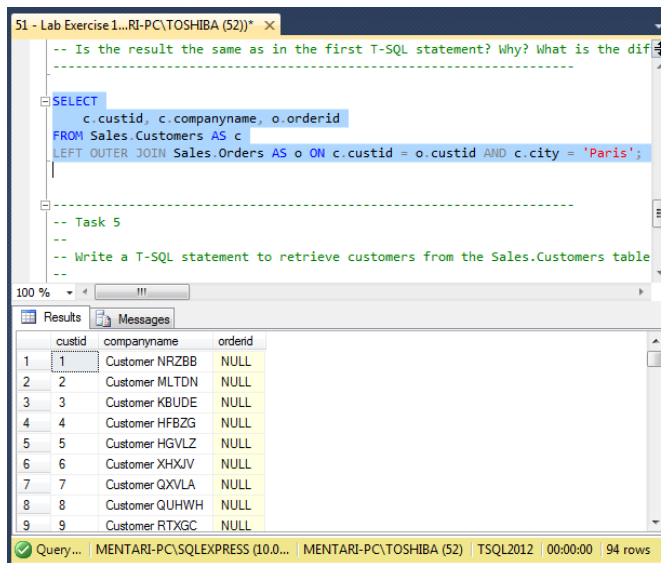
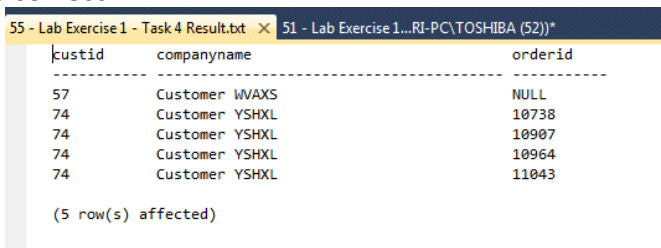
Results		Messages		
	empid	firstname	lastname	calendardate
1	1	Sara	Davis	2024-01-01
2	1	Sara	Davis	2024-01-02
3	1	Sara	Davis	2024-01-03
4	1	Sara	Davis	2024-01-04
5	1	Sara	Davis	2024-01-05
6	1	Sara	Davis	2024-01-06
7	1	Sara	Davis	2024-01-07
8	1	Sara	Davis	2024-01-08
9	1	Sara	Davis	2024-01-09

6

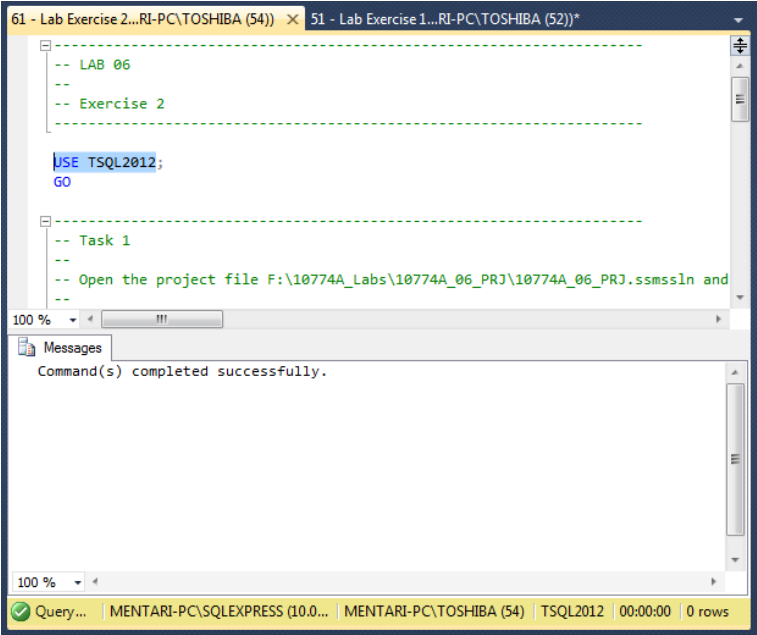
Conclusion : After completing this practical section, you will understand how to write T-SQL CROSS-JOIN code .

Practical – Part 11 : Writing Queries Who Will Filter Data with WHERE clause

Step	Information
1	The scenario in this practicum uses the problems in the marketing department. The marketing department is working on several campaigns for old customers. The marketing staff needs a different customer list according to several business rules. Therefore, <i>the developer</i> will write a SELECT command to retrieve the desired rows from the Sales.Customers table. Open the project \10774A Labs\10774A_06_PRJ\10774A_06_PRJ.ssmssln and the T-SQL script 51 - Lab Exercise 1.sql. Make sure the database is connected with "TSQL". 
2	Write a SELECT statement that will return the column values from a table, Then filter the results to only customers who are from "Brazil"! <pre> SELECT custid , companyname , contactname , address , city , country , telephone FROM Sales . Customers WHERE country = Brazil' ; </pre> Use of the N prefix for literal characters (N'Brazil'). This prefix is used because the country column is a Unicode data type. When expressing Unicode characters literally, the N character (for National) is specified as the prefix.
3	[Question- 27] Execute the 2nd stage of the test and compare it with the file 52 - Lab Exercise 1 - Task 1 Result.txt . If the results are the same, then your test is correct.

4	[Question- 28] Write a SELECT command that will return values in the custid, companyname, contactname, address, city, columns. country, and phone in the Sales.Customers table , then filter the results only for “Brazil, UK and USA” (Use the IN predicate in the WHERE clause).
5	[Question-2 9] Execute the 3rd stage test and compare it with file 53 - Lab Exercise 1 - Task 2 Result.txt . If the results are the same, then your test is correct.
6	The IT department has written T-SQL code to return values in the custid, companyname columns in the Sales.Customers table and the orderid column. in the Sales.Orders table as below: <pre>SELECT c . custid , c . companyname , o . orderid FROM Sales . Customers AS c LEFT OUTER JOIN Sales . Orders AS o ON c . custid = o . custid AND c . city = 'Paris' ;</pre>
7	Query execution in the 7th stage of the trial. Note two things, first the query will retrieve all rows in the Sales.Customers table . Second, the use of the comparison operator with the ON clause makes the city column more specific, namely the same as the value "Paris". 
8	[Question-30] Copy the T-SQL Code in step 7 then modify it with the comparison operator for the city column in the WHERE clause. After that execute the code, show the result!
9	Compare the results of step 9 with file 55 - Lab Exercise 1 - Task 4 Result.txt . If the results are the same, then your test is correct. 
10	Conclusion : After completing the practicum and answering the questions in this section, you should understand how to filter data rows from one or more tables using the WHERE clause with logical operator predicates.

Practical – Part 11 : Writing Queries Which Will Sort Data with clause ORDER BY

Step	Information
1	The case study in this lab is based on a problem in the sales department. The sales department wants to create a report that shows all orders with some customer information. In addition, there is an additional request to sort the data based on order dates and the customer IDs. The order rows in the previous lab were displayed without using the ORDER BY clause, therefore specifically for this lab section the WHERE command will be followed by the ORDER BY clause. Open the project \10774A Labs\10774A_06_PRJ\10774A_06_PRJ.ssmssln and the T-SQL script 61 - Lab Exercise 2.sql . Make sure the database is connected with "TSQL". 
2	[Question- 31] Write a SELECT command to retrieve the custid, custname columns from the Sales.Customers table and theorderid, orderdate columns from the Sales.Orders table ! Filter the results only for orders on or after April 1, 2008. Then sort the results based on orderdate in descending order and custid in ascending order!
	[Question- 32] Execute the 2nd stage of the test and compare it with the file 62 - Lab Exercise 2 - Task 1 Result.txt . If the results are the same, then your test is correct.



61 - Lab Exercise 2...A-27\stema-30 (51))*

```
SELECT
    c.custid,
    c.contactname AS custname,
    o.orderid,
    o.orderdate
FROM
    Sales.Customers AS c
INNER JOIN
    Sales.Orders AS o
ON
    c.custid = o.custid
WHERE
    o.orderdate >= '2008-04-01'
ORDER BY
    o.orderdate DESC,
    c.custid ASC;
```

100 %

Results Messages

	custid	custname	orderid	orderdate
1	9	Raghav, Amritansh	11076	2008-05-06 00:00:00.000
2	65	Moore, Michael	11077	2008-05-06 00:00:00.000
3	68	Myrcha, Jacek	11075	2008-05-06 00:00:00.000
4	73	Gonzalez, Nuria	11074	2008-05-06 00:00:00.000
5	20	Kane, John	11072	2008-05-05 00:00:00.000
6	44	Louverdis, George	11070	2008-05-05 00:00:00.000
7	46	Dressler, Marlies	11071	2008-05-05 00:00:00.000
8	58	Fakhouri, Fadi	11073	2008-05-05 00:00:00.000
9	17	Jones, TiAnna	11067	2008-05-04 00:00:00.000
10	62	Misiec, Anna	11068	2008-05-04 00:00:00.000
11	80	Geschwandtner, Jens	11069	2008-05-04 00:00:00.000
12	46	Dressler, Marlies	11065	2008-05-01 00:00:00.000

A-27\MYSQLZAHRA (14... | ACERSTEMA-27\stema-30 ... | TSQL | 00:00:00 | 88 rows

The T-SQL command from the previous practicum followed by the WHERE command is as follows:

3

```
SELECT
    e . empid , e . lastname , e . firstname , e . title , e . mgrid ,
    m . lastname AS mgrlastname , m . firstname AS mgrfirstname
FROM HR . Employees AS e
INNER JOIN HR . Employees AS m ON e . mgrid = m . empid
WHERE
    mgrlastname = N'Buck' ;
```



4

[Question- 33] Execute the T-SQL command at stage 3. Did an error occur? What is the error message? What do you think is the cause?

```
61 - Lab Exercise 2...A-27\stema-30 (51))* -p X
e.lastname,
e.firstname,
e.title,
e.mgrid,
m.lastname AS mgrlastname,
m.firstname AS mgrfirstname
FROM HR.Employees AS e
INNER JOIN HR.Employees AS m ON e.mgrid = m.empid
WHERE
mgrlastname = N'Buck';

-- Task 2
--
-- Notice that there are full source table names written as
--
```

100 %

Messages

Msg 207, Level 16, State 1, Line 33
Invalid column name 'mgrlastname'.

Completion time: 2024-09-11T00:58:52.5566129+07:00

Yes it is, In the WHERE clause, when I trying to filter based on the alias mgrlastname, which is defined in the SELECT clause. SQL Server does not allow using column aliases in the WHERE clause. Aliases defined in the SELECT clause are not recognized in the WHERE clause because the WHERE clause is evaluated before the SELECT clause.

[Question-3 4] Make changes to the T-SQL command to fix the error in the 3rd trial, then execute it! Compare the execution results with the file 63 - Lab Exercise 2 - Task 2 Result.txt. If the same, then the test result is correct.

63 - Lab Exercise 2 - Task 2 Result.txt

empid	lastname	firstname	title	mgrid	mgrlastname
6	Suurs	Paul	Sales Representative	5	Buck
7	King	Russell	Sales Representative	5	Buck
9	Dolgopyatova	Zoya	Sales Representative	5	Buck

(3 row(s) affected)



```
61 - Lab Exercise 2...A-27\stema-30 (51))* -> X
SELECT
    e.empid,
    e.lastname,
    e.firstname,
    e.title,
    e.mgrid,
    m.lastname AS mgrlastname,
    m.firstname AS mgrfirstname
FROM
    HR.Employees AS e
INNER JOIN
    HR.Employees AS m ON e.mgrid = m.empid
WHERE
    m.lastname = N'Buck';
```

	empid	lastname	firstname	title	mgrid	mgrlastname	mgrfirstname
1	6	Suurs	Paul	Sales Representative	5	Buck	Sven
2	7	King	Russell	Sales Representative	5	Buck	Sven
3	9	Dolgopyatova	Zoya	Sales Representative	5	Buck	Sven

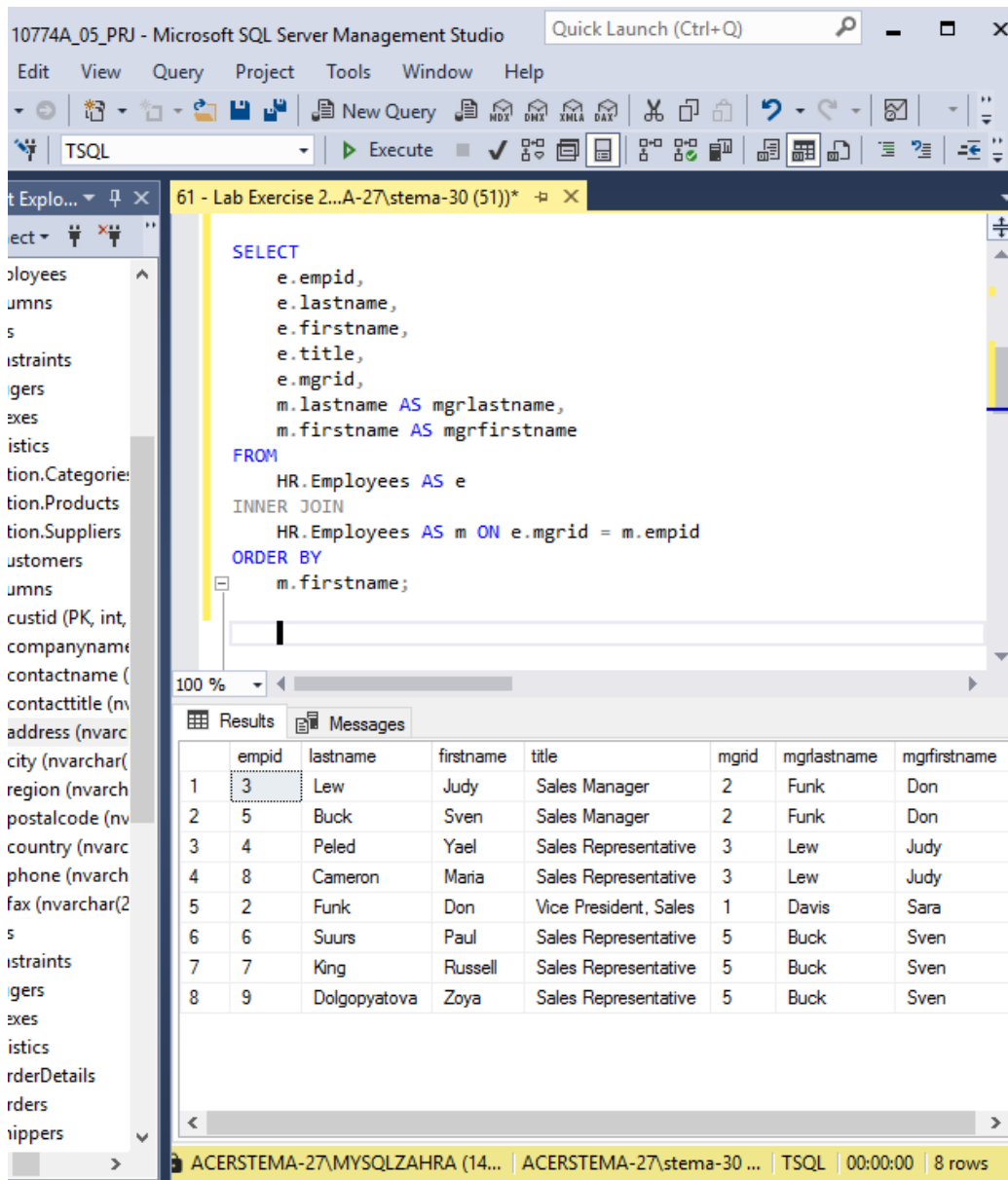
[Question- 35] Copy the T-SQL command in experiment 4, and modify it to produce all employees ORDER BY manager's first name. Initially test using the table's original name, then test using the table's alias name! Execute the T-SQL and compare the results to the 64 - Lab Exercise 2 - Task 3 Result.txt file . If the results are the same, then the experiment was correct.

64 - Lab Exercise 2 - Task 3 Result.txt

empid	lastname	firstname	title	mgrid	mgrlastname
3	Lew	Judy	Sales Manager	2	Funk
5	Buck	Sven	Sales Manager	2	Funk
4	Peled	Yael	Sales Representative	3	Lew
8	Cameron	Maria	Sales Representative	3	Lew
2	Funk	Don	Vice President, Sales	1	Davis
6	Suurs	Paul	Sales Representative	5	Buck
7	King	Russell	Sales Representative	5	Buck
9	Dolgopyatova	Zoya	Sales Representative	5	Buck

(8 row(s) affected)

[Question-3 6] Why can we use column names according to the original table name or use table alias names?



The screenshot shows the Microsoft SQL Server Management Studio interface. The query editor contains the following SQL query:

```
SELECT
    e.empid,
    e.lastname,
    e.firstname,
    e.title,
    e.mgrid,
    m.lastname AS mgrlastname,
    m.firstname AS mgrfirstname
FROM
    HR.Employees AS e
INNER JOIN
    HR.Employees AS m ON e.mgrid = m.empid
ORDER BY
    m.firstname;
```

The Results tab shows the following data:

	empid	lastname	firstname	title	mgrid	mgrlastname	mgrfirstname
1	3	Lew	Judy	Sales Manager	2	Funk	Don
2	5	Buck	Sven	Sales Manager	2	Funk	Don
3	4	Peled	Yael	Sales Representative	3	Lew	Judy
4	8	Cameron	Maria	Sales Representative	3	Lew	Judy
5	2	Funk	Don	Vice President, Sales	1	Davis	Sara
6	6	Suurs	Paul	Sales Representative	5	Buck	Sven
7	7	King	Russell	Sales Representative	5	Buck	Sven
8	9	Dolgopyatova	Zoya	Sales Representative	5	Buck	Sven

The status bar at the bottom indicates: ACERSTEMA-27\MYSQLZAHRA (14... ACERSTEMA-27\stema-30 ... TSQL 00:00:00 8 rows

1. Readability and Simplicity:

- **Table Aliases:** Make queries shorter and easier to read, especially with multiple tables or complex joins.
- **Original Table Names:** Can be more explicit and clear, especially in simpler queries or when avoiding ambiguity is crucial.

2. Avoiding Ambiguity:

- **Table Aliases:** Help distinguish between columns from different tables, especially when the same table is used multiple times in a query (e.g., self-joins).

3. Scope and Usage:

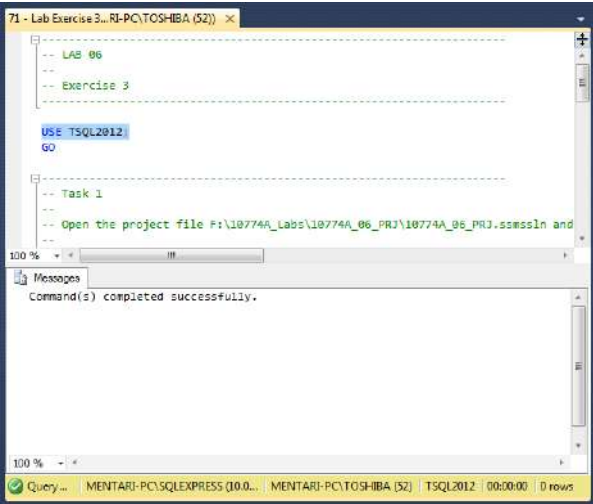
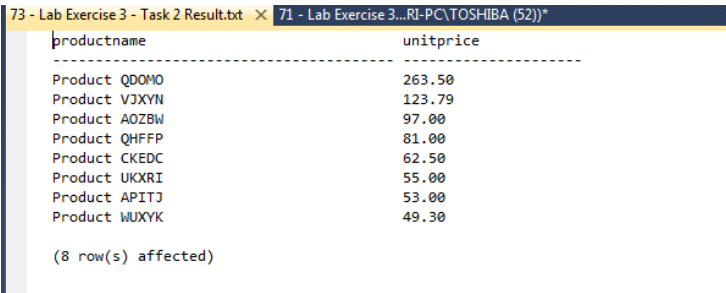
- **Table Aliases:** Are valid only within the specific query scope where they are defined.
- **Original Table Names:** Refer directly to the table and are always valid in any query context where the table is referenced.

In summary, both methods are correct, but table aliases are often preferred for their brevity and

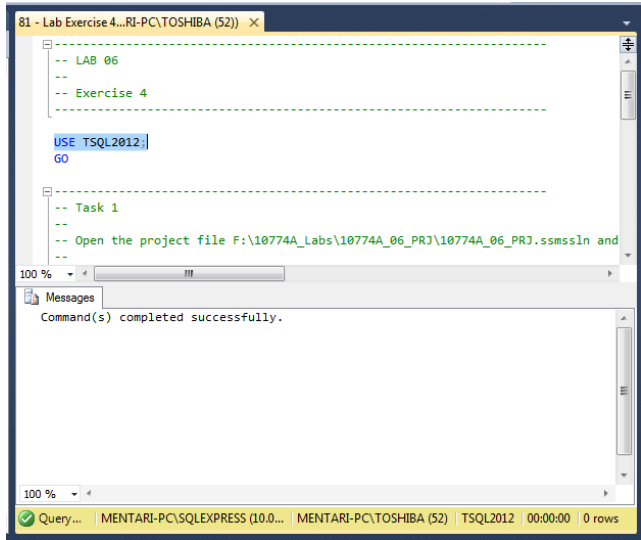
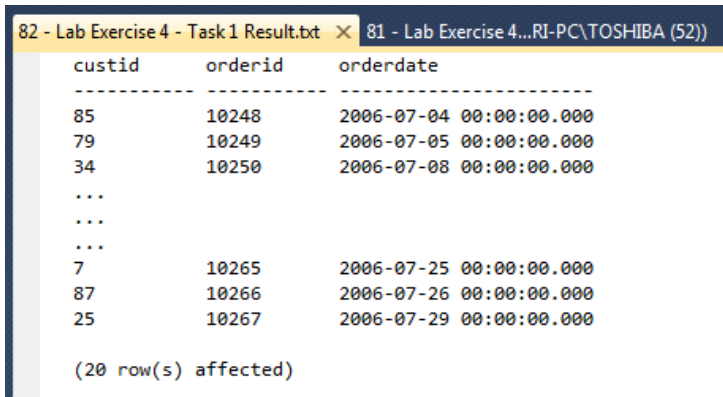


	clarity in complex queries.
7	Conclusion : After working on the practical work and questions in this section, you should now understand how to use the ORDER BY clause .

Practical – Part 12 : Writing Queries Who Will Do Data Filtering with clauses TOP

Step	Information																		
1	Part 8 of the lab uses a case study on the sales department. The sales department wants to create an additional report that shows the order invoices and the 10 percent of the most expensive products that have been sold. Open the project \10774A Labs\10774A_06_PRJ\10774A_06_PRJ.ssmssln and the T-SQL script 71 - Lab Exercise 3.sql . Make sure the database is connected with “TSQL”. 																		
2	[Question- 37] Write a SELECT command to display the productname and unitprice columns in the Production.Products table sorted descending by unitprice! Show the execution results!																		
3	[Question- 38] Copy and modify the T-SQL command in trial 2 with the limitation that only 10 percent of the child products are displayed based on unitprice ordering! Execute the command, and compare whether it is in accordance with the file 73 - Lab Exercise 3 - Task 2 Result.txt.  <table border="1"> <thead> <tr> <th>productname</th> <th>unitprice</th> </tr> </thead> <tbody> <tr><td>Product QDOMO</td><td>263.50</td></tr> <tr><td>Product VJXYN</td><td>123.79</td></tr> <tr><td>Product AOZBW</td><td>97.00</td></tr> <tr><td>Product QHFFP</td><td>81.00</td></tr> <tr><td>Product CKEDC</td><td>62.50</td></tr> <tr><td>Product UKXRI</td><td>55.00</td></tr> <tr><td>Product APITJ</td><td>53.00</td></tr> <tr><td>Product WUXYK</td><td>49.30</td></tr> </tbody> </table> (8 row(s) affected)	productname	unitprice	Product QDOMO	263.50	Product VJXYN	123.79	Product AOZBW	97.00	Product QHFFP	81.00	Product CKEDC	62.50	Product UKXRI	55.00	Product APITJ	53.00	Product WUXYK	49.30
productname	unitprice																		
Product QDOMO	263.50																		
Product VJXYN	123.79																		
Product AOZBW	97.00																		
Product QHFFP	81.00																		
Product CKEDC	62.50																		
Product UKXRI	55.00																		
Product APITJ	53.00																		
Product WUXYK	49.30																		
4	[Question- 39] Is it possible to implement the 5 trial T-SQL command using the OFFSET-FETCH clause?																		
5	Conclusion : After completing the practical work and questions in this section, you should now understand how to apply the TOP option to the SELECT clause of the T-SQL command.																		

Practical – Part 13 : Writing Queries Who Will Filter Data with OFFSET-FETCH clause

Step	Information
1	Practical part 9 will implement paging solution to display rows from Sales.Orders table , because the number of rows is too many. On each report page, user can only see 20 rows. Open the project \10774A Labs\10774A_06_PRJ\10774A_06_PRJ.ssmssln and the T-SQL script 81 - Lab Exercise 4.sql . Make sure the database is connected with “TSQL”. 
2	[Question- 40] Write a SELECT command to display the custid,orderid, and orderdate columns in the Sales.Orders table . Sort the rows by orderdate and orderid. Take the first 20 rows. Execute the command and compare the results with the file 82 - Lab Exercise 4 - Task 1 Result.txt. If the results are the same, then your test is correct. 
3	[Question- 41] Write a SELECT statement to display the same results as question no. 43, skip the first 20 rows, and continue with the next 20 rows using the OFFSET-FETCH clause! Execute the statement and compare 83 - Lab Exercise 4 - Task 2 Result.txt. If the results are the same, then your test is correct.



71 - Lab Exercise 3...A-27\stema-30 (51))*

```
-- Execute the written statement and compare the results that you get
```

```
SELECT
    productname,
    unitprice
FROM
    Production.Products
ORDER BY
    unitprice DESC;
```

```
-- Task 3
--
-- Write a SELECT statement to retrieve the productname and unitprice
--
-- Execute the T-SQL statement and notice the number of the rows returned
```

100 %

Results Messages

	productname	unitprice
1	Product QDOMO	263.50
2	Product VJXYN	123.79
3	Product AOZBW	97.00
4	Product QHFFP	81.00
5	Product CKEDC	62.50
6	Product UKXRI	55.00
7	Product APITJ	53.00
8	Product WUXYK	49.30
9	Product ZZZHR	46.00
10	Product OFBNT	45.60
11	Product SMIOH	43.90
12	Product ICKNK	43.90



	83 - Lab Exercise 4 - Task 2 Result.txt X <table><tr><th>custid</th><th>orderid</th><th>orderdate</th></tr><tr><td>33</td><td>10268</td><td>2006-07-30 00:00:00.000</td></tr><tr><td>89</td><td>10269</td><td>2006-07-31 00:00:00.000</td></tr><tr><td>87</td><td>10270</td><td>2006-08-01 00:00:00.000</td></tr><tr><td>...</td><td></td><td></td></tr><tr><td>...</td><td></td><td></td></tr><tr><td>...</td><td></td><td></td></tr><tr><td>63</td><td>10285</td><td>2006-08-20 00:00:00.000</td></tr><tr><td>63</td><td>10286</td><td>2006-08-21 00:00:00.000</td></tr><tr><td>67</td><td>10287</td><td>2006-08-22 00:00:00.000</td></tr></table> (20 row(s) affected)	custid	orderid	orderdate	33	10268	2006-07-30 00:00:00.000	89	10269	2006-07-31 00:00:00.000	87	10270	2006-08-01 00:00:00.000	...			...			...			63	10285	2006-08-20 00:00:00.000	63	10286	2006-08-21 00:00:00.000	67	10287	2006-08-22 00:00:00.000
custid	orderid	orderdate																													
33	10268	2006-07-30 00:00:00.000																													
89	10269	2006-07-31 00:00:00.000																													
87	10270	2006-08-01 00:00:00.000																													
...																															
...																															
...																															
63	10285	2006-08-20 00:00:00.000																													
63	10286	2006-08-21 00:00:00.000																													
67	10287	2006-08-22 00:00:00.000																													
4	Conclusion : After working on the practical work and questions in this section, you should now understand how to use the OFFSET-FETCH clause in T-SQL commands.																														

-- Have a great time doing it -